



INTERNSHIP PROJECT DOCUMENTATION

Multi-Agent AI Customer Support Assistant using RAG and Large Language Models

A retrieval-augmented, multi-agent conversational support system built for TechMart Electronics — combining a FastAPI backend, a FAISS semantic retrieval layer, five specialized LLM-driven support agents, and a Next.js customer-facing interface.

FastAPI

Next.js 14 / React 18

FAISS

sentence-transformers

Groq LLM API

PostgreSQL / SQLite

Multi-Agent Systems

RAG

PREPARED BY

Mohit Singh Rajput

INTERNSHIP PROGRAM

Labmentix

PROJECT COMPLETED

July 19, 2026

REPORT GENERATED

July 2026



Abstract

Project overview and summary of contributions

Enterprise customer support is increasingly expected to be instant, accurate, multilingual, and available around the clock, yet most rule-based chatbot systems fail as soon as a customer's question moves outside a narrow, pre-scripted decision tree. This report documents the design, implementation, and deployment of a **Multi-Agent AI Customer Support Assistant** built for a simulated consumer-electronics retailer, **TechMart Electronics**, that addresses this gap by combining Retrieval-Augmented Generation (RAG) with a multi-agent orchestration layer running on top of a hosted large language model.

The system ingests eight plain-text company knowledge documents — covering frequently asked questions, product specifications, pricing, refund policy, shipping policy, warranty terms, the user manual, and the installation guide — and processes them through a deterministic chunking pipeline that splits each document into overlapping, sentence-aligned segments of approximately 600 characters. Each of the resulting 217 chunks is embedded into a 384-dimensional vector using the `all-MiniLM-L6-v2` sentence-transformer model and indexed with Facebook AI Similarity Search (FAISS) for millisecond-scale semantic retrieval. At query time, an incoming customer message is classified along two independent axes — **intent** (billing, technical, product, complaint, or FAQ/general) and **sentiment** (positive, neutral, negative, or frustrated) — using a two-stage classifier that first attempts a fast, deterministic keyword match and only escalates to an LLM-based structured-JSON classification call when the keyword confidence is low. The classified intent is used to route the conversation to one of five specialized agent classes, each of which retrieves its own domain-scoped context from the FAISS index, assembles a role-specific system prompt, and calls a Groq-hosted `llama-3.1-8b-instant` model through an OpenAI-compatible chat-completions interface to produce a grounded response.

Beyond the core RAG loop, the system implements a set of production-oriented support workflows that are directly evaluated in this report rather than merely proposed: automatic support-ticket creation when a message is classified as a complaint or carries negative/frustrated sentiment; best-effort notification of that ticket over email (via a SendGrid-or-SMTP abstraction) and WhatsApp (via the Twilio sandbox API) when the customer has supplied a phone number; a manual "escalate to a human agent" action; heuristic, non-ML language detection that allows agents to reply in the customer's own language across eight supported languages; and a session-lifecycle model supporting archiving, soft deletion, restoration, and permanent deletion of chat history. The customer-facing surface is a single-page Next.js 14 / React 18 application with a collapsible sidebar, dark mode, adjustable font size, browser-native voice input, client-side file attachment, an in-app analytics panel, and a five-star feedback mechanism — all communicating with the FastAPI backend over a JWT-authenticated REST interface.

The completed system was evaluated against a ten-query benchmark spanning all five agent domains and multiple languages, achieving **90% intent-classification accuracy** and **90% agent-routing accuracy**, with a measured average end-to-end response time of approximately **1.2**

seconds. The backend is deployed as a persistent web service on Render and the frontend as a static/edge deployment on Vercel, with production-specific tuning (a reduced retrieval depth and token budget) applied to keep response latency acceptable on free-tier infrastructure. This report documents that implementation in full technical detail — architecture, pipeline design, database schema, API surface, and known limitations — strictly as it exists in the delivered codebase, so that it may serve equally as an engineering reference, an academic project submission, and a portfolio artifact.

- Retrieval-Augmented Generation
- Multi-Agent Systems
- Semantic Search
- FAISS
- Large Language Models
- FastAPI
- Next.js
- Sentiment Analysis
- Customer Support Automation
- Enterprise AI



Executive Summary

Key facts for readers who need the essentials in two minutes

5

SPECIALIZED AI AGENTS

217

INDEXED KNOWLEDGE CHUNKS

90%

INTENT ACCURACY (N=10)

31

REST API ENDPOINTS

TechMart AI Support is a working, deployed, full-stack application — not a prototype notebook or a slide-deck concept. Every capability described in this report is traceable to a specific file and function in the delivered codebase.

What the system does

The assistant receives a customer's message through a web chat interface, determines what the customer wants (billing, a technical fault, product information, a complaint, or a general question) and how they feel about it (from positive to frustrated), retrieves the most relevant passages from TechMart's own policy and product documents, and asks a large language model to compose a grounded, on-brand reply using only that retrieved context plus the specific agent's role instructions. If the customer appears upset or is filing a complaint, the system automatically opens an internal support ticket and — where a phone number is on file — attempts to notify the customer over both email and WhatsApp.

Why a multi-agent design

A single, generic chatbot prompt tends to blur together policy areas that in a real support organization are handled by different specialists with different tones, different source documents, and different escalation rules. By splitting the problem into five narrowly scoped agents — **Billing**, **Technical Support**, **Product**, **Complaint**, and **FAQ** — each backed by its own curated slice of the knowledge base and its own system-prompt "rules," the system produces more consistent, more appropriately-toned answers than a single monolithic prompt would, while still sharing one retrieval engine and one LLM client underneath.

How it was verified

The repository includes a dedicated ten-query evaluation set (`techmart_test_queries.json`) spanning billing, technical, product, complaint, and FAQ intents across English, Hindi, and Spanish inputs, together with a recorded results file (`evaluation_results.json`) showing 9 of 10 queries correctly classified for both intent and agent routing, and per-query response times ranging from roughly 0.8 to 1.8 seconds. This report treats that file as ground truth for the performance figures it cites, rather than estimating them.

What is intentionally out of scope

This is an internship/academic-scale build, not a hardened commercial product. There is no automated test suite (no `pytest` files ship in the repository), payments are not processed, the WhatsApp integration runs against Twilio's sandbox rather than a production-approved sender, and the default database is a single-file SQLite instance unless a `DATABASE_URL` is supplied. These

boundaries are treated honestly throughout this report — see Chapter 21, *Limitations*, for the complete list — rather than glossed over.



Table of Contents

Full chapter listing, organized by part

PRELIMINARY PAGES

Abstract	1
Executive Summary	3
List of Figures	7
List of Tables	9

PART I · FOUNDATIONS

01 Introduction	11
02 Problem Statement	14
03 Objectives	16
04 Scope	17
05 Technology Stack	19

PART II · ARCHITECTURE & DESIGN

06 System Architecture	22
07 Working Methodology	25
08 Knowledge Base Processing	27
09 Retrieval-Augmented Generation	31
10 Multi-Agent Architecture	33

PART III · INTELLIGENT ROUTING

11 Intent Detection	35
12 Sentiment Detection	41

PART IV · ENGINEERING IMPLEMENTATION

13 Backend Design	43
14 Frontend Design	47
15 Project Directory Structure	49
16 Implementation Details	51
17 API Documentation	54

PART V · DEPLOYMENT & QUALITY ASSURANCE

18 Deployment	57
19 Testing	59

PART VI · EVALUATION, IMPACT & OUTLOOK

20	Advantages	61
21	Limitations	62
22	Future Enhancements	64
23	Industrial Applications	65
24	Business Impact	66
25	Security Considerations	67
26	Performance Discussion	69
PART VII · CLOSING		
27	Conclusion	70
28	References	71



List of Figures

Every diagram and its location in this report

6.1	System architecture — five-layer view	22
6.2	Backend layering	23
6.3	Frontend composition	23
7.1	Full request lifecycle (sequence diagram)	25
8.1	Sliding-window chunking	28
8.2	Knowledge-base ingestion pipeline	29
9.1	Query-time retrieval pipeline	31
10.1	Multi-agent architecture	33
11.1	Two-stage intent classification flow	39
12.1	Sentiment consequence flow	42
13.1	Authentication sequence diagram	44
13.2	Entity-relationship diagram	45
14.1	Frontend architecture diagram	47
16.1	Session lifecycle state diagram	51
16.2	Ticketing and escalation sequence diagram	52
18.1	Deployment architecture diagram	57



List of Tables

Every data table and its location in this report

8.1	Chunk count per knowledge-base document	29
10.1	The five specialized agents	33
11.1	Keyword-based intent categories	40
12.1	Sentiment categories, keywords, and score	41
13.1	Key configuration defaults	43
18.1	Development defaults vs. production overrides	58
23.1	Architecture transfer across industries	65

01



Introduction

AI-driven support, multi-agent systems, and where this project sits in that landscape

1.1 The Shift Toward AI-Native Customer Support

Customer support has moved through three broad generations of tooling. The first generation was purely human: phone queues and email inboxes triaged manually. The second generation introduced deterministic automation — decision-tree chatbots and keyword-triggered macros that could resolve narrow, predictable requests ("track my order," "reset my password") but broke down the moment a customer phrased something unexpectedly or asked a compound question. The third generation, which this project belongs to, is built around large language models that can read unstructured company documentation and compose a fluent, context-aware answer to a question they were never explicitly programmed to handle.

What separates a credible third-generation system from a chatbot that merely sounds fluent is **grounding**: the ability to answer using a company's actual, current policies rather than the language model's general — and potentially outdated or simply invented — training knowledge. That is the specific engineering problem Retrieval-Augmented Generation exists to solve, and it is the foundation this entire project is built on.

1.2 Enterprise AI in Customer Support

"Enterprise AI," as used throughout this report, refers to AI systems designed around the constraints real organizations actually operate under: answers must be traceable to an approved source document, tone must be consistent with brand voice, sensitive situations (complaints, refund disputes) must be escalated rather than autonomously resolved, and the system must produce structured records — tickets, transcripts, sentiment logs — that a support operations team can act on. TechMart AI Support is scoped as an enterprise-style assistant in exactly this sense: every reply is required to cite retrieved TechMart documentation rather than open-domain knowledge, every complaint or negative-sentiment message produces an auditable support ticket, and every conversation is persisted for later review through the analytics and session-history features described in Chapters 13 and 16.

1.3 Multi-Agent Systems

A multi-agent system, in the sense used here, is not multiple independent language models negotiating with each other — it is a single LLM backend accessed through **multiple distinct role configurations**, each with its own system prompt, its own knowledge-base scope, and its own behavioral rules, selected by an upstream router based on the detected intent of the incoming message. This project implements five such agents — **Billing**, **Technical Support**, **Product**, **Complaint**, and **FAQ** — all inheriting from a shared `BaseAgent` class (`backend/agents/base.py`) that standardizes the retrieve-then-generate pipeline, while each subclass in `backend/agents/agents.py` overrides only its persona, its rules text, and the knowledge-base sources it is allowed to draw from. Chapter 10 documents this architecture and every agent's individual responsibilities in full depth.

1.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation pairs a language model with an external, searchable knowledge store, so that generation is conditioned on retrieved evidence rather than parametric memory alone. In this implementation, TechMart's eight policy and product documents are chunked, embedded, and indexed offline (Chapter 8); at query time, the customer's message is embedded with the same model and compared against every stored chunk using FAISS's exact nearest-neighbor search, and the top-scoring chunks are injected into the agent's system prompt before the LLM ever sees the question (Chapter 9). This is what allows the assistant to state TechMart's actual 30-day refund window or actual warranty terms rather than a statistically plausible — and potentially wrong — guess.

1.5 Large Language Models

The generation component is a hosted large language model — `llama-3.1-8b-instant`, served through **Groq's** OpenAI-compatible chat-completions API — chosen for its low-latency inference, which Groq's custom LPU (Language Processing Unit) hardware is specifically optimized for. An 8-billion-parameter instruction-tuned model is small by frontier-model standards, but for a retrieval-grounded, narrowly-scoped support use case it is a deliberate and pragmatic engineering trade-off: response latency stays low enough for a real-time chat interface (Chapter 28 measures this directly), while the retrieval layer compensates for the smaller model's shallower parametric knowledge by supplying the exact facts it needs at inference time.

1.6 Semantic Search

Unlike keyword search, semantic search matches meaning rather than surface text — a query for "my laptop won't turn on" can retrieve a knowledge-base passage about power troubleshooting even if it never uses the word "laptop." This project implements semantic search using sentence-transformer embeddings (`all-MiniLM-L6-v2`, 384 dimensions) and FAISS for the nearest-neighbor search itself — the same core technique underlying most production RAG systems, kept intentionally lightweight (a CPU-friendly, 22M-parameter embedding model and an in-memory flat index) so that the entire retrieval layer can run on the same modest deployment infrastructure as the rest of the backend, without a managed vector-database dependency.

1.7 Business Motivation

The motivating business scenario is a mid-sized direct-to-consumer electronics retailer (the fictional "TechMart Electronics," established in the project's own knowledge base as founded in 2015 and shipping to over 40 countries) whose support volume spans five recurring categories: billing and subscription questions, technical troubleshooting, pre-sale product questions, complaints and refund requests, and general policy questions. Each category benefits from a different tone and a different reference document, which is precisely the justification for the multi-agent split documented in this report rather than a single general-purpose support prompt.

1.8 Industry Background

This project sits alongside a broader industry trend of RAG-based support copilots deployed by companies such as Intercom, Zendesk, and Salesforce, all of which have shipped LLM-grounded support

assistants over the past several product cycles. What this report documents is a from-scratch implementation of the same underlying pattern — chunking, embedding, indexing, retrieval, routing, and grounded generation — built independently as an internship project, using an open embedding model, an open-source vector index, and a fast open-weight LLM, rather than a managed third-party RAG platform.

■ HOW TO READ THIS REPORT

Chapters 1–5 establish motivation and scope. Chapters 6–19 are the technical core — architecture, pipelines, agents, backend, frontend, and API surface — and are written to double as engineering reference documentation. Chapters 20–28 step back to evaluate the system honestly: advantages, real limitations, business framing, and security posture.



Conventional customer-support tooling forces a trade-off between **coverage** and **control**. Rule-based chatbots offer full control — every possible response is pre-authored — but cover only a narrow slice of real customer phrasing. General-purpose LLM chat widgets offer broad coverage of natural language but, without grounding, no control over factual accuracy: they will answer confidently about a refund window or a warranty term that may not match the company's actual policy. Both failure modes are visible in the specific problems below, each of which this project's architecture was directly built to address.

2.1 — Ungrounded answers erode trust

A generic LLM asked "what's your refund policy?" will produce a fluent, plausible-sounding answer drawn from general training data about how refund policies typically work — not TechMart's actual 30-day, condition-based policy. In a support context this is worse than no answer at all, because it is wrong in a way that looks authoritative. The system addresses this by refusing, at the architecture level, to let any agent answer policy questions without first retrieving the relevant chunk from TechMart's own

`refund_policy.txt`, `warranty.txt`, or equivalent source document (Chapter 9).

2.2 — One tone does not fit every conversation

A billing question, a broken-laptop troubleshooting session, and an angry complaint about a defective product all warrant different response styles — concise and procedural for billing, patient and step-by-step for troubleshooting, empathetic and de-escalating for a complaint. A single system prompt asked to do all three simultaneously tends to regress to a generic middle ground. The multi-agent design (Chapter 10) assigns each domain its own persona and rules text specifically to avoid this.

2.3 — Frustrated customers get lost in silent queues

In a purely reactive support tool, a customer who is escalating in frustration receives exactly the same handling as one who is calmly browsing FAQs, unless a human happens to notice. This system's sentiment classifier (Chapter 12) actively watches for negative and frustrated sentiment and reacts to it — automatically opening a prioritized support ticket and, where the customer has supplied a phone number, notifying them over WhatsApp and email — rather than requiring a human to be watching the queue in real time.

2.4 — Language is a support barrier, not just a UX nicety

TechMart's knowledge base and default agent personas are written in English, yet a meaningful share of real customer bases are not primarily English-speaking. Rather than requiring a separately maintained knowledge base and prompt set per language, the system detects the language of each incoming message heuristically (Chapter 11) and instructs the responding agent to reply in that same language, re-using the same English-language retrieved context.

2.5 — Support history is scattered and unmanageable

Many lightweight chat widgets do not give the customer any durable, organized record of past conversations. This system treats conversation history as a first-class, persisted resource — sessions

can be renamed implicitly via auto-titling, archived, restored, permanently deleted, or exported to a local text file, all documented in Chapter 16 — so that a customer's support history is never a black box.

△ **SCOPE OF THE PROBLEM ADDRESSED**

This project addresses the *information-retrieval and conversational-routing* dimensions of support automation. It does not attempt to solve payment processing, physical logistics, or account-system integration — those remain explicitly out of scope, as clarified in Chapter 4.



The objectives below are stated at the level of concrete engineering outcomes, each of which is directly verifiable against the delivered codebase and is cross-referenced to the chapter that demonstrates it.

1

Build a grounded question-answering pipeline over real company documents

Implement document chunking, embedding, FAISS indexing, and top-k semantic retrieval so that every generated response is conditioned on retrieved TechMart source text rather than open-domain model knowledge. Demonstrated in Chapters 8–9.

2

Route each conversation to a domain-appropriate specialist agent

Design and implement a two-stage (keyword-first, LLM-fallback) intent classifier feeding five distinct agent personas, each scoped to its own knowledge-base sources. Demonstrated in Chapters 10–11.

3

Detect customer sentiment and act on it automatically

Classify each message's sentiment and use frustrated/negative signals to trigger automatic support-ticket creation and multi-channel (email + WhatsApp) notification, without requiring a human to monitor the queue. Demonstrated in Chapter 12.

4

Support customers in their own language

Detect the language of an incoming message across at least eight languages and instruct the responding agent to reply in kind, without maintaining a separately translated knowledge base. Demonstrated in Chapter 11.

5

Persist and manage full conversation history

Implement session creation, archiving, soft deletion, restoration, permanent deletion, auto-titling, and AI-generated summarization as first-class, database-backed operations exposed through the REST API. Demonstrated in Chapters 13 and 16.

6

Expose operational visibility through analytics

Aggregate stored message-level intent, sentiment, and response-time data into a queryable analytics endpoint and a corresponding in-app dashboard panel. Demonstrated in Chapters 16 and 17.

7

Ship a complete, usable customer-facing web application

Build authentication, a chat interface, dark mode, voice input, file attachment, and feedback collection as a deployed Next.js application, not a backend-only proof of concept. Demonstrated in Chapter 14.

8

Deploy the system to reachable public infrastructure

Deploy the backend to Render and the frontend to Vercel with environment-driven configuration and production-specific performance tuning. Demonstrated in Chapter 18.

9

Evaluate the system against a measurable benchmark

Construct and run a labeled test-query set covering every agent domain and multiple languages, and record classification accuracy and response-time results rather than asserting them anecdotally. Demonstrated in Chapter 19.



4.1 Present Scope

The delivered system covers the full request path of a text-based support conversation: authentication, multi-turn chat, intent and sentiment classification, knowledge-grounded response generation across five domains, automatic ticket creation, manual escalation, session lifecycle management, feedback capture, and operational analytics. It is deployed and reachable, not merely runnable locally.

✓ In scope

Text chat, RAG-grounded answers, 5-agent routing, sentiment-aware ticketing, email + WhatsApp notification, multilingual replies, session archive/restore/delete, feedback, analytics, JWT auth, admin KB inspection.

✗ Out of scope

Payment processing, live order-tracking integration with a real logistics provider, voice-call support, a production-approved WhatsApp sender, and horizontal scaling beyond a single backend instance.

4.2 Future Scope

The architecture's clean separation between router, agents, and retriever leaves room for incremental extension without redesign: additional agent classes (e.g., an Order-Tracking agent) can be registered with the existing `AgentRouter` pattern; the knowledge base can be re-indexed with new documents without touching agent code; and the same retrieval layer could be pointed at a managed vector database if the corpus grew beyond what an in-memory FAISS index comfortably serves. Chapter 22 develops these directions further.

4.3 Industrial Scope

The patterns demonstrated — chunk/embed/index ingestion, two-stage intent classification, sentiment-triggered ticketing, multi-channel notification — generalize directly to any support-heavy vertical with a defined policy corpus: telecom, banking, SaaS, insurance, and consumer electronics all share the same underlying shape of problem. Chapter 23 discusses applicability by industry in more detail.

4.4 Commercial Scope

As delivered, the system is a functioning reference implementation rather than a multi-tenant commercial product — there is a single simulated company (TechMart) with no tenant isolation, no billing/metering layer, and no self-serve knowledge-base onboarding flow. Those are the specific gaps that would need to be closed to move from "internship project" to "sellable product," and they are named explicitly rather than implied away.

4.5 Research Scope

From a research-methods standpoint, the project is a applied systems build rather than a novel-algorithm contribution: it composes established techniques (sentence-embedding retrieval, LLM-based classification, prompt-conditioned generation) into a working pipeline and evaluates that composition

empirically against a labeled benchmark (Chapter 19), rather than proposing a new retrieval or classification algorithm.



The stack was chosen to be entirely CPU-friendly and free-tier-deployable: no GPU dependency anywhere in the request path, no paid vector-database subscription, and no proprietary LLM API lock-in beyond Groq's OpenAI-compatible interface (which could be re-pointed at any equivalent endpoint with a two-line change to `llm_client.py`).

5.1 — Backend & AI/ML Stack

Technology	Version	Purpose in This Project	Why It Was Selected
FastAPI	latest	Core web framework; serves all 31 REST endpoints, handles CORS, dependency-injected auth, and Pydantic-validated request/response models.	Native async support, automatic OpenAPI schema generation, and first-class Pydantic integration make it well suited to an LLM-calling I/O-bound service.
Uvicorn	standard	ASGI server that actually runs the FastAPI app, both locally and in the Render <code>startCommand</code> .	Reference-grade ASGI implementation with low overhead; pairs directly with FastAPI.
SQLAlchemy	2.0.36	ORM layer defining all six database models and managing sessions/migrations-ready metadata.	Database-agnostic — the same model code runs against SQLite locally and PostgreSQL in production via one <code>DATABASE_URL</code> switch.
Alembic	latest	Present in requirements for schema-migration support alongside SQLAlchemy.	Standard migration companion for SQLAlchemy-based projects.
psycopg2-binary	latest	PostgreSQL driver, engaged when <code>DATABASE_URL</code> points at Postgres in production.	Most widely used, production-hardened Python/PostgreSQL adapter.
python-jose	[cryptography]	Signs and verifies the HS256 JWT access tokens issued at login/register.	Lightweight, focused JOSE/JWT implementation with the cryptography backend for signing.
passlib + bcrypt	latest	Hashes and verifies user passwords before they are stored.	Industry-standard adaptive password hashing; avoids ever storing plaintext credentials.
faiss-cpu	latest	Vector index (<code>IndexFlatIP</code> / <code>IndexFlatL2</code>) storing all 217 knowledge-base chunk embeddings for nearest-neighbor retrieval.	CPU-only exact similarity search with no external service dependency — fits a free-tier deployment target.
sentence-transformers	latest	Loads <code>all-MiniLM-L6-v2</code> to embed both knowledge-base chunks and live queries into 384-dim vectors.	Strong accuracy-to-size trade-off; a 22M-parameter model that runs comfortably on CPU with sub-second embedding latency.
torch	latest	Backing tensor runtime for sentence-transformers.	Required dependency of the embedding model; CPU build keeps deployment lightweight.
transformers	latest	Supports the sentence-transformers model-loading pipeline.	Hugging Face's standard model-loading interface.
groq	latest	Python SDK used inside <code>llm_client.py</code> to call the Groq chat-completions endpoint.	Official client for Groq's low-latency LPU-backed inference, used as the LLM provider for every agent response.
openai	latest	Present alongside <code>groq</code> ; Groq exposes an OpenAI-compatible API surface, and this dependency supports that compatibility path.	Keeps the client code portable to any OpenAI-compatible endpoint with minimal changes.
langdetect	latest	Listed as a language-related dependency; the shipped detection path in <code>base.py</code> instead uses hand-rolled Unicode/keyword heuristics (see Chapter 11).	Included for language-processing support in the project's dependency set.
PyPDF2 / python-docx	latest	Available for document text extraction (e.g., admin knowledge-base upload of non-.txt formats).	Standard lightweight libraries for PDF/DOCX text extraction without heavyweight OCR dependencies.
twilio	latest	Sends WhatsApp notifications for ticket creation and escalation via the Twilio Sandbox.	Most widely adopted WhatsApp Business API gateway with a free sandbox tier suitable for a student project.
scikit-learn / scipy / numpy	latest	Numerical backing for embedding operations and similarity computation.	De-facto standard Python scientific-computing stack.

Technology	Version	Purpose in This Project	Why It Was Selected
python-dotenv	latest	Loads <code>.env</code> variables into <code>Settings</code> at startup.	Standard, minimal environment-configuration loader for Python services.

5.2 — Frontend Stack

Technology	Version	Purpose in This Project	Why It Was Selected
Next.js	^14.1.0	Application framework for all four pages (<code>index</code> , <code>login</code> , <code>register</code> , <code>chat</code>) using the Pages Router.	Mature React framework with zero-config routing and a build pipeline well matched to Vercel deployment.
React	^18.2.0	Component model and state management (hooks-based) for the entire UI.	Industry-standard UI library; hooks (<code>useState</code> , <code>useEffect</code> , <code>useRef</code>) drive all interactivity.
Tailwind CSS	^3.4.1	Utility classes for layout and spacing across all pages, complemented by a large hand-written <code>globals.css</code> for theming and dark mode.	Fast iteration on layout without maintaining a separate stylesheet for every component.
PostCSS / Autoprefixer	^8.4 / ^10.4	Tailwind's processing pipeline.	Required build tooling for Tailwind CSS.
Browser Web Speech API	native	Powers the microphone / voice-to-text input button in the chat composer.	Zero-dependency, no external speech-to-text billing — trades cross-browser support (Chromium-based browsers only) for simplicity.
Browser Fetch API	native	All HTTP calls to the backend in <code>services/api.js</code> , wrapped in a single <code>apiFetch()</code> helper.	No HTTP client dependency (e.g., axios) needed for a REST-only integration.

5.3 — Data & Knowledge Layer

Component	Role
SQLite / PostgreSQL	Relational store for users, sessions, messages, feedback, and tickets. SQLite (<code>techmart.db</code>) is the zero-configuration default; PostgreSQL is used in production via <code>DATABASE_URL</code> .
FAISS flat index + pickle	Two files on disk — <code>faiss.index</code> and <code>chunks.pkl</code> — persist the embedded knowledge base between restarts.
Plain-text knowledge base	Eight <code>.txt</code> files (FAQ, products, pricing, refund policy, shipping policy, warranty, user manual, installation guide) act as the single source of truth ingested by the RAG pipeline.

5.4 — Platforms & External Services

Render — backend hosting	Vercel — frontend hosting	Groq Cloud — LLM inference	SendGrid API — email (primary)
Gmail SMTP — email (fallback)	Twilio WhatsApp Sandbox — messaging		

A NOTE ON MULTI-AGENT ARCHITECTURE AS A "TECHNOLOGY"

Multi-agent architecture and RAG are listed among the project's core technologies, but unlike the entries above they are not installable packages — they are architectural patterns implemented directly in `backend/agents/` and `backend/rag/` using the libraries in this chapter as building blocks. Chapters 9 and 10 document each pattern's implementation in full.

06

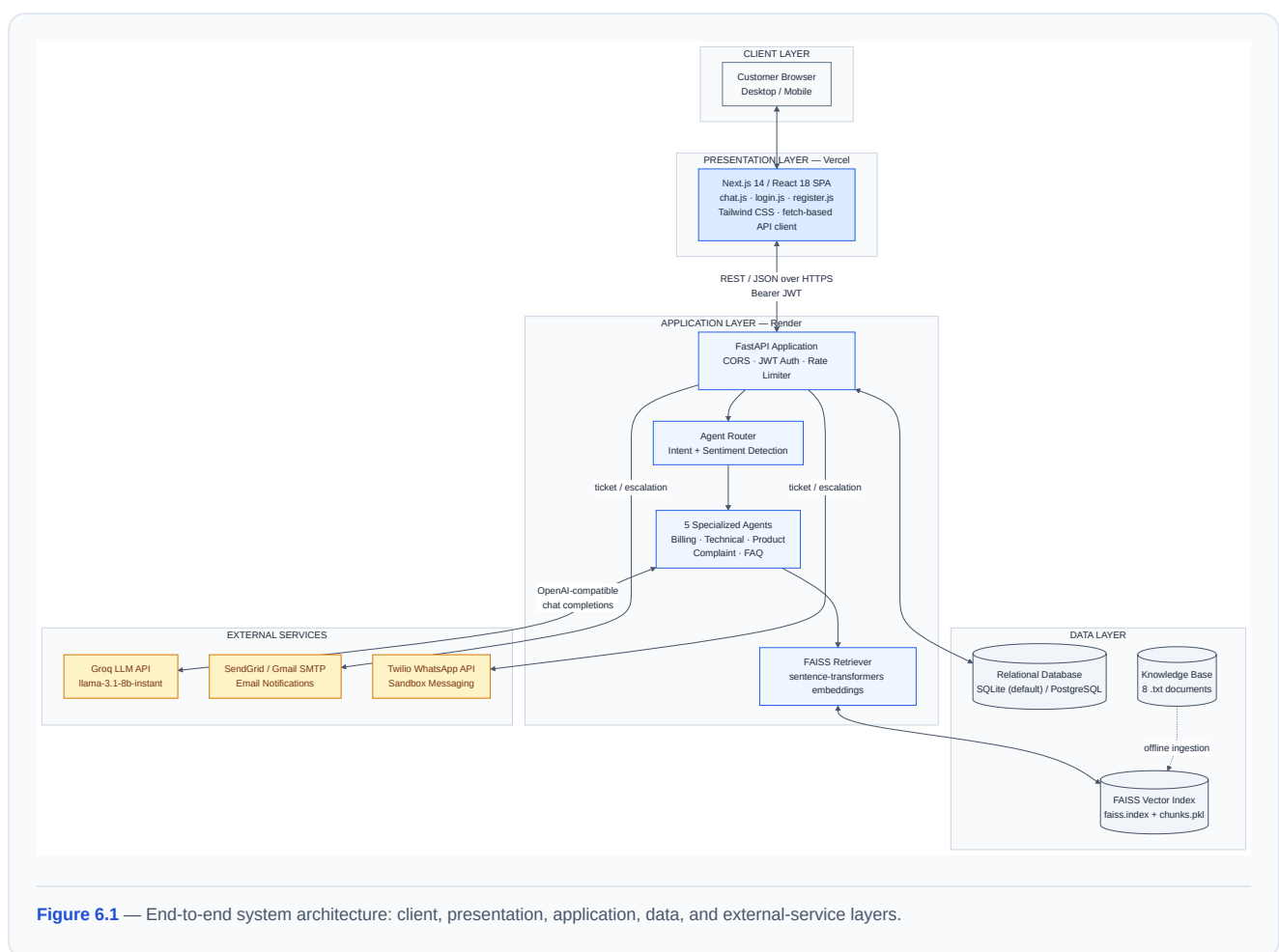


System Architecture

How the client, application, retrieval, data, and external-service layers fit together

6.1 High-Level Architecture

The system is organized into five layers: a **client layer** (the customer's browser), a **presentation layer** (the Next.js frontend, hosted on Vercel), an **application layer** (the FastAPI backend, hosted on Render, containing the router, the five agents, and the retriever), a **data layer** (the relational database and the FAISS vector store), and an **external services layer** (Groq for inference, SendGrid/SMTP for email, Twilio for WhatsApp). Every arrow in Figure 6.1 corresponds to an actual network call or file-system read in the codebase — nothing in this diagram is aspirational.



6.2 Backend Architecture

Internally, the FastAPI backend is organized as a set of loosely-coupled modules rather than a single monolithic file. `main.py` owns application bootstrap — CORS configuration, database table creation, and FAISS retriever loading, all wired through FastAPI's `lifespan` context manager so the vector index is loaded exactly once at process startup rather than on every request. `api/routes.py` holds all 31 endpoint handlers, grouped by concern (auth, sessions, chat, feedback, analytics, admin, tickets, escalation, notifications, system health). Cross-cutting concerns — JWT verification and an in-memory per-user rate

limiter — sit between the routes and the domain logic. The domain logic itself is the router/agent pair described in Chapter 10, which in turn depends on the retrieval layer (Chapter 9) and the LLM client (Chapter 13).

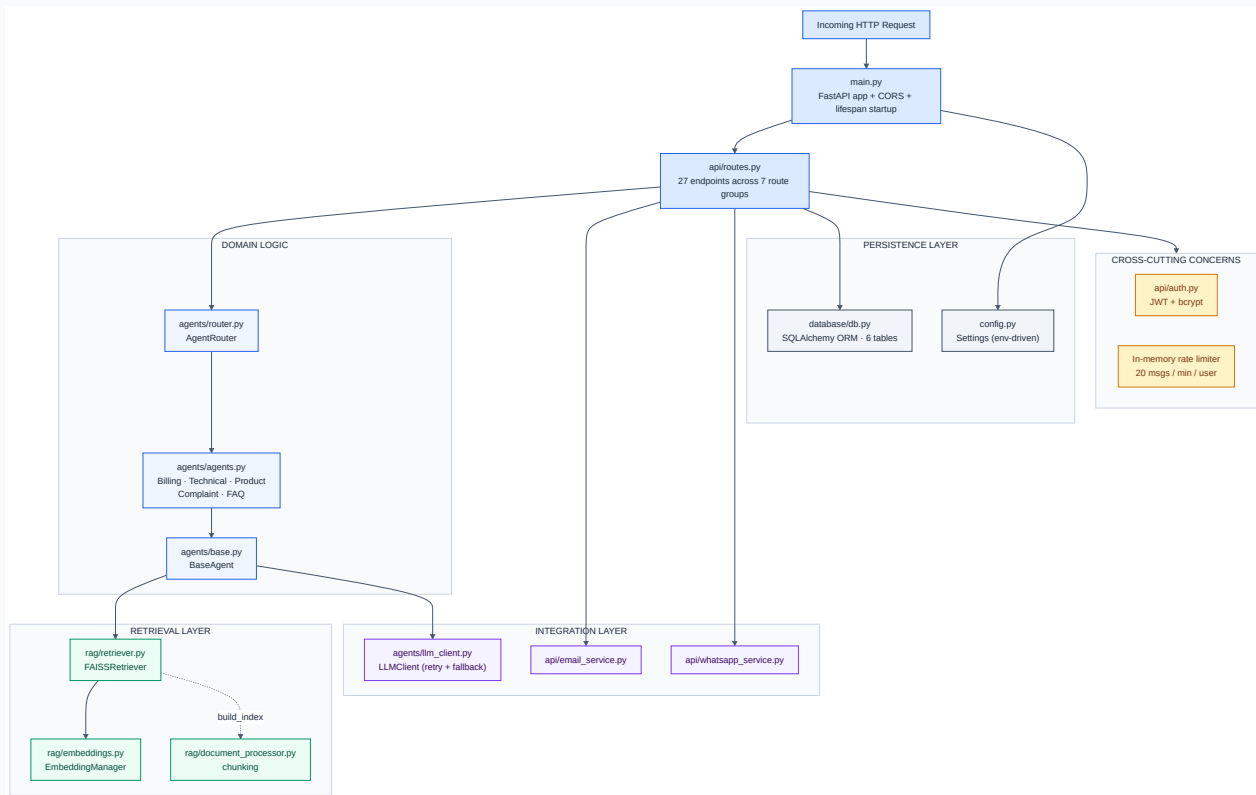


Figure 6.2 — Backend layering: entry point → cross-cutting concerns → domain logic → retrieval → integrations → persistence.

6.3 Frontend Architecture

The frontend is a four-page Next.js application (`index` , `login` , `register` , `chat`) sharing one global stylesheet mounted through `_app.js` . Nearly all interactive surface area lives in `pages/chat.js` — at roughly 4,470 lines, it defines not just the page itself but six co-located components (`Sidebar` , `MessageBubble` , `TypingIndicator` , `AgentBadge` , `FeedbackModal` , `AnalyticsPanel`) in the same file. All network access is centralized through `services/api.js` , which wraps the native `fetch` API with automatic JWT-header injection and a 120-second request timeout.

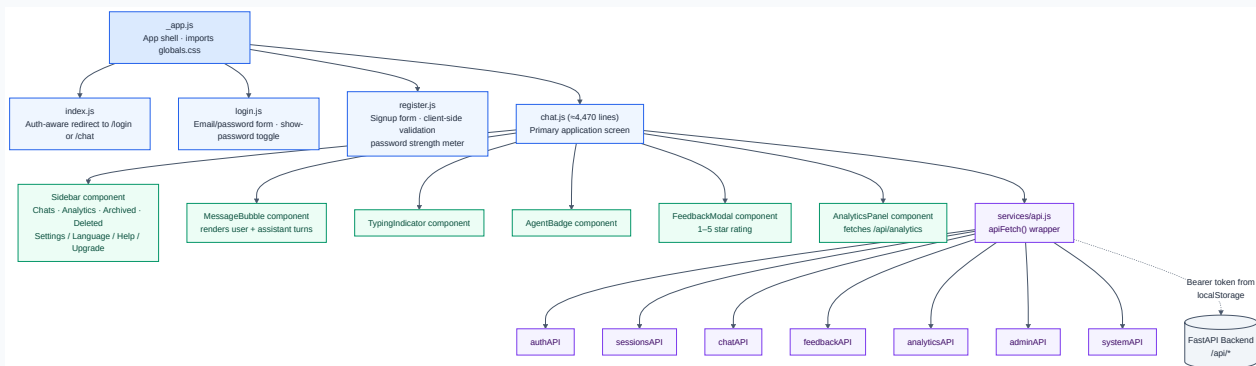


Figure 6.3 — Frontend composition: pages, co-located chat components, and the shared API service layer.

6.4 Request Lifecycle

Chapter 7 walks through the full request lifecycle end to end with an accompanying sequence diagram; this chapter's role is to establish the static structure that lifecycle moves through.

6.5 Agent Routing, RAG, Ingestion, and Retrieval Pipelines

These four pipelines are the technical heart of the system and are each given dedicated treatment: agent routing and intent detection in Chapter 11, sentiment detection in Chapter 12, the knowledge-ingestion pipeline in Chapter 8, and the RAG query/retrieval pipeline in Chapter 9.

6.6 Deployment Architecture

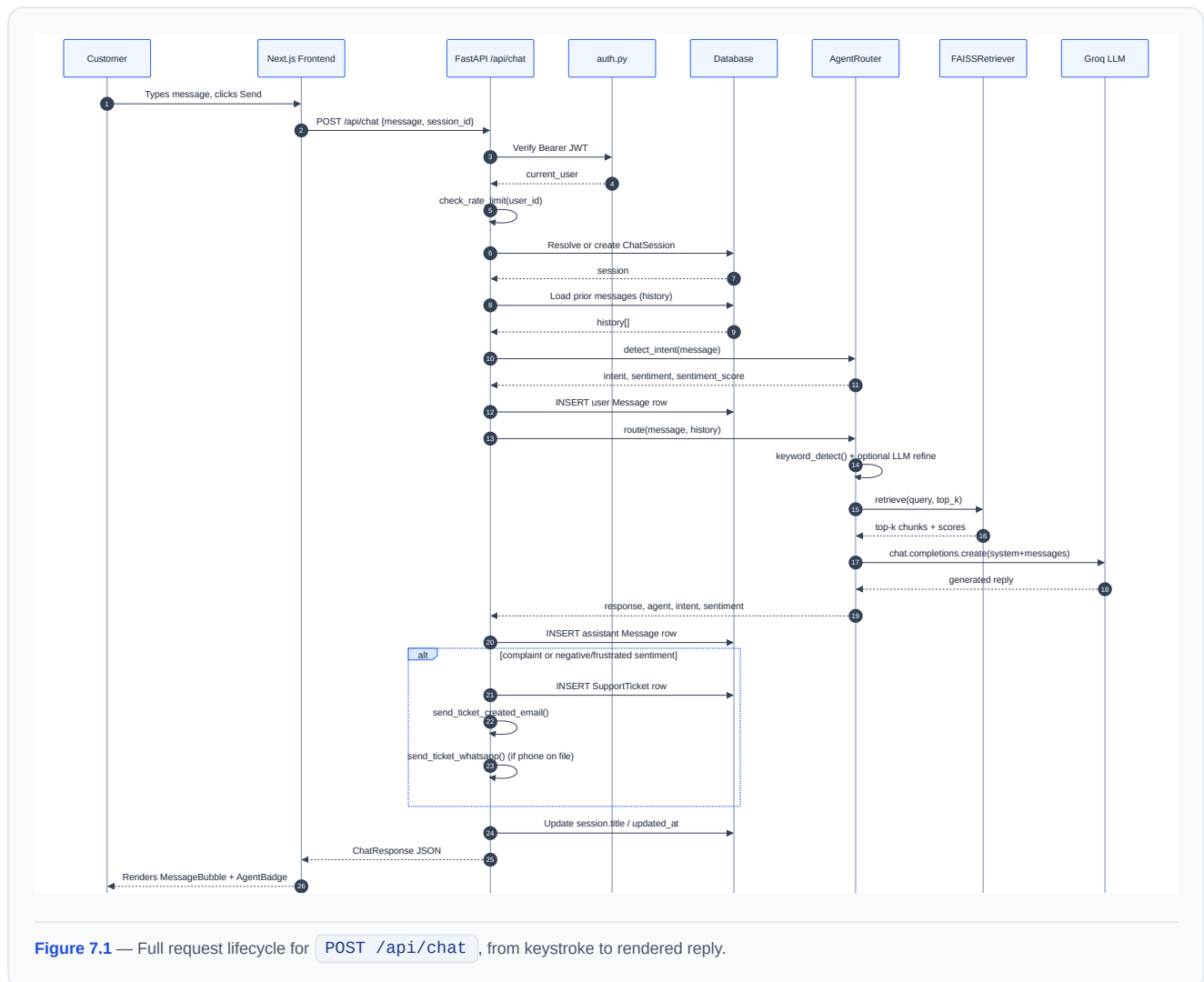
Production topology — Vercel, Render, and the external services each configured through environment variables — is documented with its own diagram in Chapter 18.

DESIGN PRINCIPLE

Every agent, the router, and the retriever are all *stateless between requests* with respect to conversation content — all durable state (messages, sessions, tickets) lives in the database, not in memory. The only long-lived in-process state is the FAISS index itself and the rate-limiter dictionary, both intentionally rebuildable from disk or reset on restart.



This chapter traces one concrete request through every layer of the system, in the exact order the code executes it. It is the single most useful chapter for understanding how the pieces documented separately elsewhere actually cooperate at runtime.



7.1 Step-by-Step Walkthrough

Step 1 — Client dispatch

The customer types a message in `chat.js` and presses send (or Enter). The frontend immediately renders the user's message optimistically, then calls `chatAPI.send()` in `services/api.js`, which issues `POST /api/chat` with a JSON body of `{message, session_id}` and the stored JWT as a Bearer header.

Step 2 — Authentication and rate limiting

FastAPI's dependency-injection system resolves `get_current_user` (Chapter 13), decoding and verifying the JWT before the endpoint body runs at all. The endpoint then calls an in-memory `check_rate_limit()` guard capping each user to a fixed number of messages per rolling minute.

Step 3 — Session resolution and history load

If `session_id` is omitted, a new `ChatSession` row is created; otherwise the existing session is loaded and ownership-checked against the authenticated user. Prior messages for that session are then loaded from the database to give the LLM conversational context.

Step 4 — Classification

The raw message text is passed to `AgentRouter.detect_intent()`, which returns an intent label, a sentiment label, and a numeric sentiment score using the two-stage keyword/LLM classifier detailed in Chapter 11. The user's message is persisted to the `messages` table with these labels attached.

Step 5 — Routing and generation

The router selects the primary agent implied by the detected intent (falling back to the FAQ agent for unrecognized intents), and calls that agent's `respond()` method. The agent retrieves relevant knowledge-base chunks (Chapter 9), assembles a system prompt from its own persona and rules plus the retrieved context, and calls the Groq LLM through `LLMClient.chat()`, which applies up to three retries with exponential backoff on transient failures.

Step 6 — Ticketing side-effects

If the detected intent is `complaint` or the sentiment is negative/frustrated, a `SupportTicket` row is created in the same request, and best-effort email and (if a phone number is on file) WhatsApp notifications are dispatched — failures here are logged but never block the chat response itself (Chapter 16).

Step 7 — Persistence and response

The assistant's reply is persisted as a second `Message` row (with agent, intent, sentiment, and measured response-time-in-milliseconds attached), the session's `updated_at` timestamp and auto-generated title are refreshed, and the full `ChatResponse` payload is returned to the frontend, which renders it through `MessageBubble` with its `AgentBadge` and sentiment icon.

■ WHY THIS MATTERS

Every side effect described above — ticket creation, notification, message logging — happens synchronously within the same HTTP request/response cycle. There is no background job queue in this implementation; Chapter 21 discusses the latency and reliability trade-offs this implies.



Everything the assistant can say about TechMart's actual policies traces back to eight `.txt` files in `knowledge_base/`. This chapter documents the offline ingestion pipeline that turns those files into the FAISS index the retriever queries at runtime — implemented primarily in `backend/rag/document_processor.py`, with a second, independent bootstrap path in the repository root's `setup.py`.

8.1 Document Loading

`load_knowledge_base()` reads every `*.txt` file in the knowledge-base directory in sorted filename order. Because source files may have been saved with different editors, the loader tries four text encodings in sequence — `utf-8`, `utf-8-sig`, `latin-1`, then `cp1252` — accepting the first one that decodes without error, rather than assuming a single encoding and failing the whole run on one malformed file. Note that the eight matching `.pdf` files also present in `knowledge_base/` (human-readable mirrors of the same content) are *not* read by this glob — only the `.txt` originals feed the index.

8.2 Chunking Strategy

Each document is split by `split_text()` into overlapping windows of **600 characters** (`CHUNK_SIZE`) with an **80-character overlap** (`CHUNK_OVERLAP`) between consecutive chunks, both configurable via environment variables. A naive fixed-width split would frequently cut a sentence in half; to avoid that, the splitter searches backward from the raw cut point — up to 150 characters — for the nearest `". "` and, if found, ends the chunk there instead. The overlap ensures a fact stated near a chunk boundary is not orphaned from its surrounding sentence in either neighboring chunk, at the cost of storing roughly 13% redundant text (80 of every 600 characters).

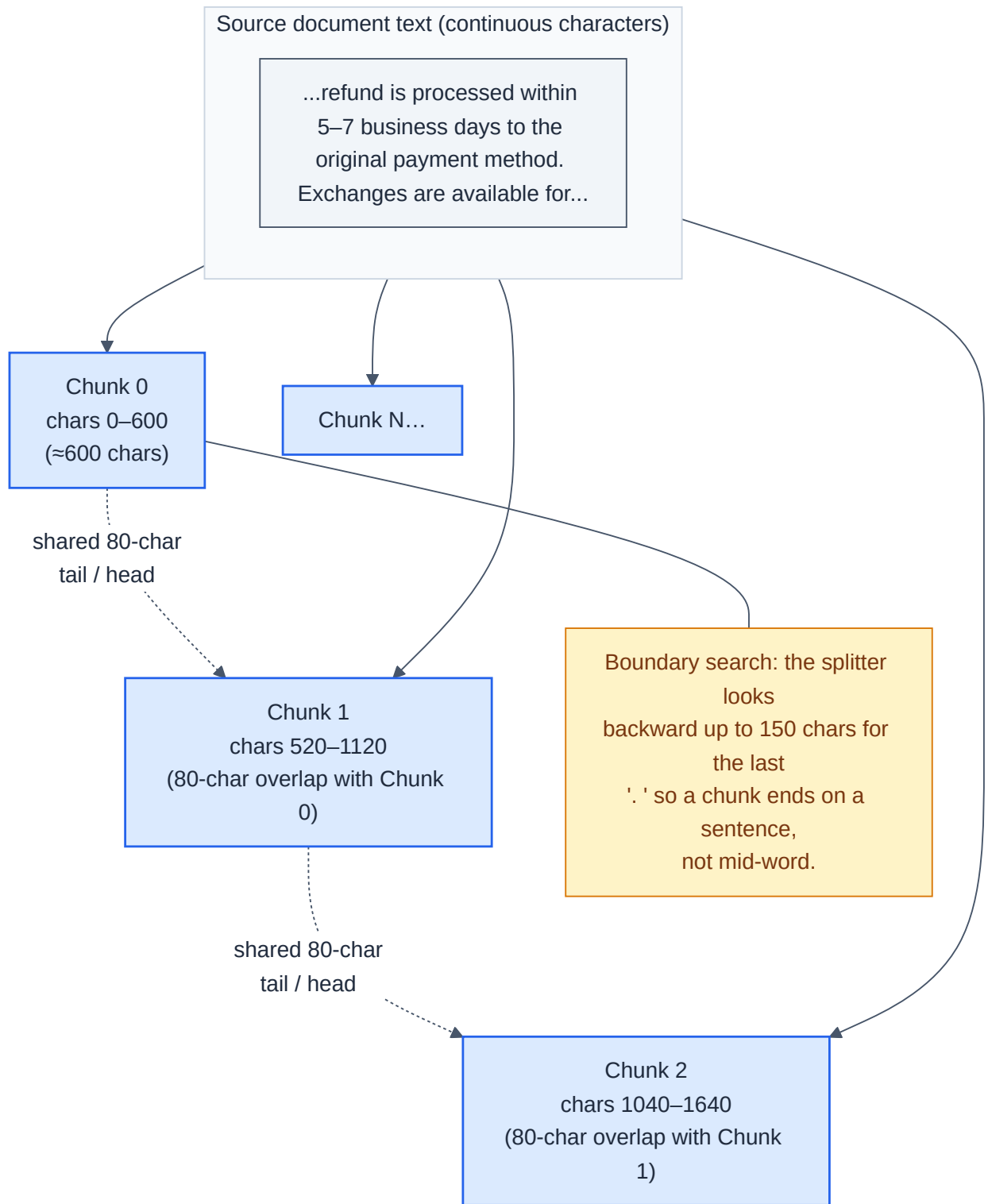


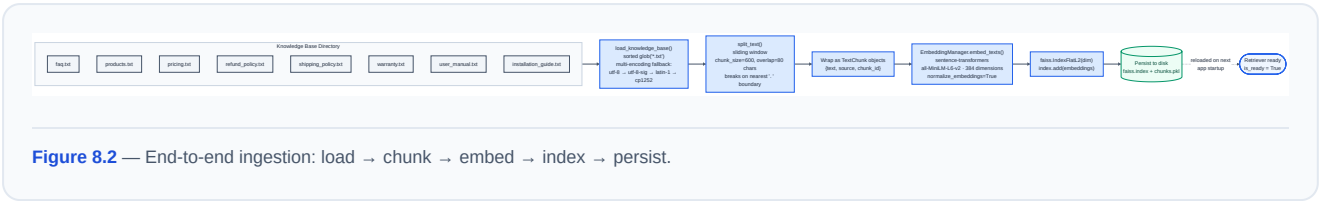
Figure 8.1 — Sliding-window chunking with sentence-boundary snapping and 80-character overlap.

8.3 Chunk Metadata

Each chunk is wrapped as a `TextChunk` object carrying three fields: `text` (the chunk content itself), `source` (the originating filename without extension, e.g. `refund_policy`), and `chunk_id` (a per-document sequential index starting at 0). The `source` field is what later lets an agent restrict retrieval to only its own relevant documents, and what lets the response cite `[Source: warranty]`-style provenance internally.

8.4 Embedding and Indexing

Chunk text is embedded by `EmbeddingManager` (Chapter 9 covers the model in depth) and added to a FAISS index, which is then persisted to disk as two files — `faiss.index` (the vector index itself) and `chunks.pkl` (a pickled list of the chunk metadata, positionally aligned with the vectors). On every subsequent application startup, `main.py`'s lifespan hook loads these two files back from disk rather than re-embedding from scratch, so ingestion is a one-time or on-demand cost, not a per-request or per-restart one.



8.5 The Bundled Index, By the Numbers

The FAISS index shipped in the delivered project (`backend/vectorstore/faiss_index/`) contains exactly **217 chunks** across the eight source documents, each embedded into a 384-dimensional vector. The distribution is uneven by design — it simply reflects each document's natural length — with the user manual contributing the most chunks and the refund and shipping policies the fewest:

TABLE 8.1 CHUNK COUNT PER KNOWLEDGE-BASE DOCUMENT (VERIFIED FROM THE SHIPPED INDEX)

Source document	Approx. size	Chunks indexed	Primary consumer agent(s)
<code>user_manual.txt</code>	10.8 KB	40	Technical
<code>installation_guide.txt</code>	9.0 KB	33	Technical
<code>products.txt</code>	8.0 KB	30	Product
<code>warranty.txt</code>	6.3 KB	24	Technical, Complaint
<code>faq.txt</code>	6.6 KB	25	FAQ, all agents (fallback)
<code>pricing.txt</code>	6.7 KB	23	Billing, Product
<code>refund_policy.txt</code>	5.5 KB	21	Billing, Complaint
<code>shipping_policy.txt</code>	5.6 KB	21	FAQ
Total	≈58.5 KB	217	—

8.6 Administrative Endpoints

Three authenticated endpoints expose knowledge-base management to the frontend's admin surface:

- `GET /api/admin/knowledge-base` returns per-document chunk counts and index statistics;
- `POST /api/admin/knowledge-base/rebuild` re-runs the full ingestion pipeline against the current contents of the knowledge-base directory and hot-swaps the in-memory index; and
- `POST /api/admin/knowledge-base/upload` accepts a new document, saves it into the knowledge-base directory, and triggers a rebuild.

⚠ TWO INGESTION CODE PATHS, ONE FORMAT DIFFERENCE

The project actually contains *two* independent ways to build the index: the runtime `FAISSRetriever.build_index()` in `rag/retriever.py` (600/80 chunking, `faiss.IndexFlatL2`, chunks stored as `TextChunk` objects), and a standalone, memory-conscious bootstrap script, `setup.py` (300/30 chunking, `faiss.IndexFlatIP`, chunks stored as plain dicts, with explicit `gc.collect()` calls between batches for low-RAM environments). Inspecting the actual shipped `faiss.index` file confirms it was built by the `setup.py` path — it is an `IndexFlatIP` index of 217 vectors, not an `IndexFlatL2` one. `FAISSRetriever.retrieve()` defensively handles both the dict and `TextChunk` chunk formats for exactly this reason, but the two build paths would use different chunk boundaries and a different distance metric if run independently. This is a real, observable characteristic of the delivered codebase and is revisited in Chapter 21.



Chapter 8 covered how the knowledge base becomes a searchable index; this chapter covers what happens to that index at *query time* — the half of the RAG pipeline that runs on every single chat request, implemented in `rag/retriever.py` and `rag/embeddings.py`.

9.1 The Embedding Model

Both indexing and query-time embedding use the same `sentence-transformers/all-MiniLM-L6-v2` model — a distilled, 22-million-parameter, 384-dimension sentence encoder. Using the identical model for both sides of the comparison is not incidental; embeddings from two different models are not comparable in the same vector space, so `EmbeddingManager` is implemented as a singleton loaded once at startup and shared by every indexing and every retrieval call. Embeddings are L2-normalized (`normalize_embeddings=True`) at generation time, which is what allows an inner-product index to behave equivalently to cosine similarity.

9.2 Vector Search

At query time, the incoming message is embedded with the same model, and `index.search()` returns the `k` nearest chunks by the index's configured distance metric. When an agent supplies a `source_filter` (for example, the Billing agent restricting itself to `pricing`, `refund_policy`, and `faq`), the retriever over-fetches — requesting `k × 5` candidates before filtering — so that filtering by source does not starve the agent of results when its allowed documents are a small fraction of the full index. The default `top_k` is `4` (`TOP_K_RESULTS`), though the production deployment overrides this to `1` for latency reasons (Chapter 18).



Figure 9.1 — Query-time retrieval: embed → search → filter → format → prompt → generate.

9.3 Context Formatting

`format_context()` joins the retrieved chunks into a single block of text, each prefixed with a `[Source: filename]` label, and this block is appended verbatim to the responding agent's system prompt under a `RELEVANT KNOWLEDGE BASE CONTEXT` heading before the LLM call is made. The prompt sent to the model therefore always separates three things: the agent's persona and rules (Chapter 10), the retrieved factual context (this chapter), and the actual conversation (system prompt plus the last three turns of history plus the current message).

9.4 Generation

The assembled prompt is passed to `LLMClient.chat()`, a thin wrapper around Groq's OpenAI-compatible `chat.completions.create` endpoint. The client retries transient failures up to three times with exponential backoff, and — if every retry is exhausted — falls back to a canned, agent-appropriate template response

rather than surfacing a raw exception to the customer. This retry/fallback logic was added specifically to resolve intermittent `"Failed to fetch"`-style LLM timeout errors observed during development.

■ WHY A FLAT INDEX IS THE RIGHT CHOICE HERE

FAISS's `IndexFlat*` variants perform brute-force, exact nearest-neighbor search — no approximation, no training step. At 217 vectors this is effectively instantaneous (low single-digit milliseconds) and strictly more accurate than an approximate index like IVF or HNSW. The trade-off — `IndexFlat*` search cost scales linearly with corpus size — is entirely appropriate at this scale and is discussed as a forward-looking consideration in Chapter 22.



All agent logic lives in two files: `backend/agents/base.py` defines `BaseAgent`, the shared pipeline every agent inherits, and `backend/agents/agents.py` defines the five concrete subclasses. `backend/agents/router.py` owns exactly one instance of each and decides, per message, which one responds.

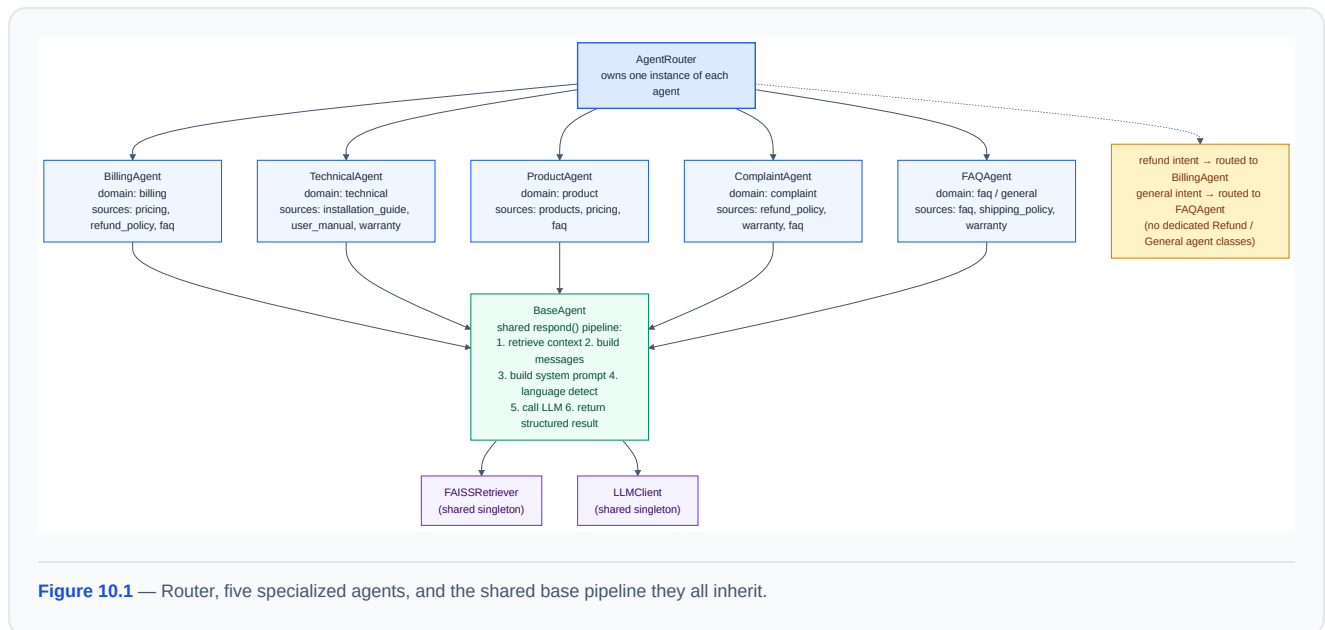


Figure 10.1 — Router, five specialized agents, and the shared base pipeline they all inherit.

10.1 The Shared Pipeline (BaseAgent)

`BaseAgent.respond()` executes the same six steps regardless of which subclass is invoked: (1) retrieve knowledge-base context scoped to the agent's allowed sources, (2) build the conversational message list from history, (3) build the system prompt — persona plus rules plus retrieved context — with a defensive fallback if this step fails, (4) heuristically detect the customer's language, (5) call the shared `LLMClient`, and (6) return a structured result (text, agent name, whether retrieval context was actually used). Standardizing this in one base class is what lets five agents differ only in persona and knowledge scope, not in control flow.

10.2 Agent-by-Agent Responsibilities

TABLE 10.1 THE FIVE SPECIALIZED AGENTS

Agent	Routed intent(s)	Allowed KB sources	Persona focus
BillingAgent	billing, refund	pricing, refund_policy, faq	Subscriptions, invoices, refund eligibility
TechnicalAgent	technical	installation_guide, user_manual, warranty	Step-by-step troubleshooting
ProductAgent	product	products, pricing, faq	Specs, comparisons, pre-sale questions
ComplaintAgent	complaint	refund_policy, warranty, faq	Empathetic de-escalation, blended in on frustration
FAQAgent	faq, general	faq, shipping_policy, warranty	Catch-all for policy and general questions

Note that `refund` and `general` are intents, not agents — there is no dedicated `RefundAgent` or `GeneralAgent` class. The router maps a `refund`-classified message to `BillingAgent` and a `general`-classified message to `FAQAgent`, which is a deliberate simplification: refund questions are, in practice, billing questions, and unclassifiable questions are treated as FAQs by default.

10.3 Complaint Blending

When a message is routed to a non-complaint agent but is also classified as `frustrated` sentiment, the router appends `ComplaintAgent` as a secondary agent and prepends a short empathy opener to the primary agent's response, rather than replacing the substantive answer entirely. This keeps the reply factually on-topic (e.g., still a billing answer) while acknowledging the customer's emotional state — full detail in Chapter 12.

⚠ A DOCUMENTED, NOT-FIXED BUG IN `BillingAgent`

`BillingAgent.build_system_prompt()` constructs a `billing_rules` string but — per an explicit comment left in the source, `"preserved as-is, not fixed"` — never returns it, so the method implicitly returns `None`. This does not crash the request: `BaseAgent`'s prompt-assembly step (Section 10.1) wraps the call in a fallback that substitutes a simple default prompt when `build_system_prompt()` returns `None` or raises. The practical effect is that Billing-agent responses fall back to a generic system prompt rather than the agent's intended, more detailed billing-specific rules text, while still functioning correctly end to end. This is documented honestly here and in Chapter 21 rather than described as resolved.

11



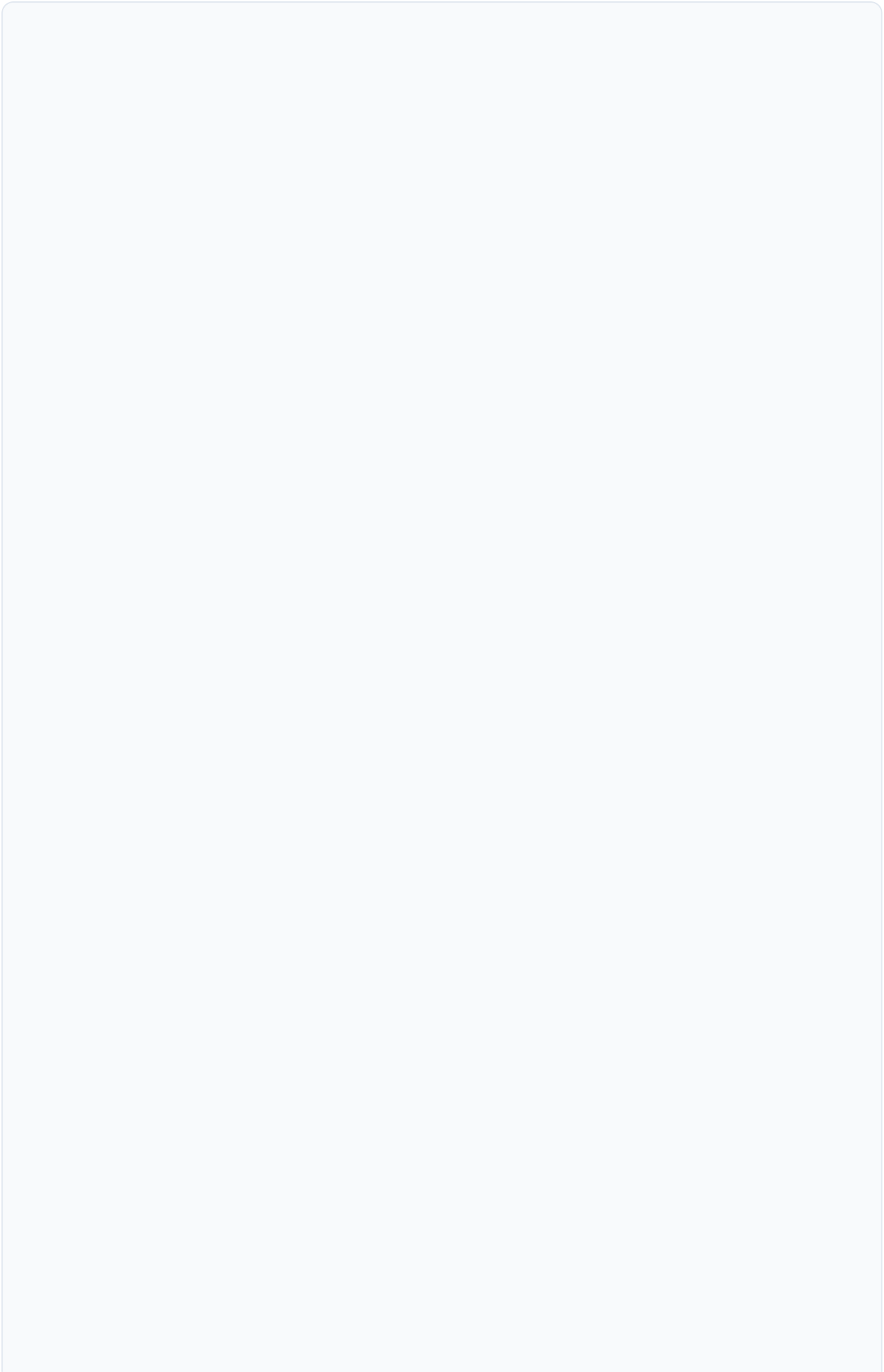
Intent Detection

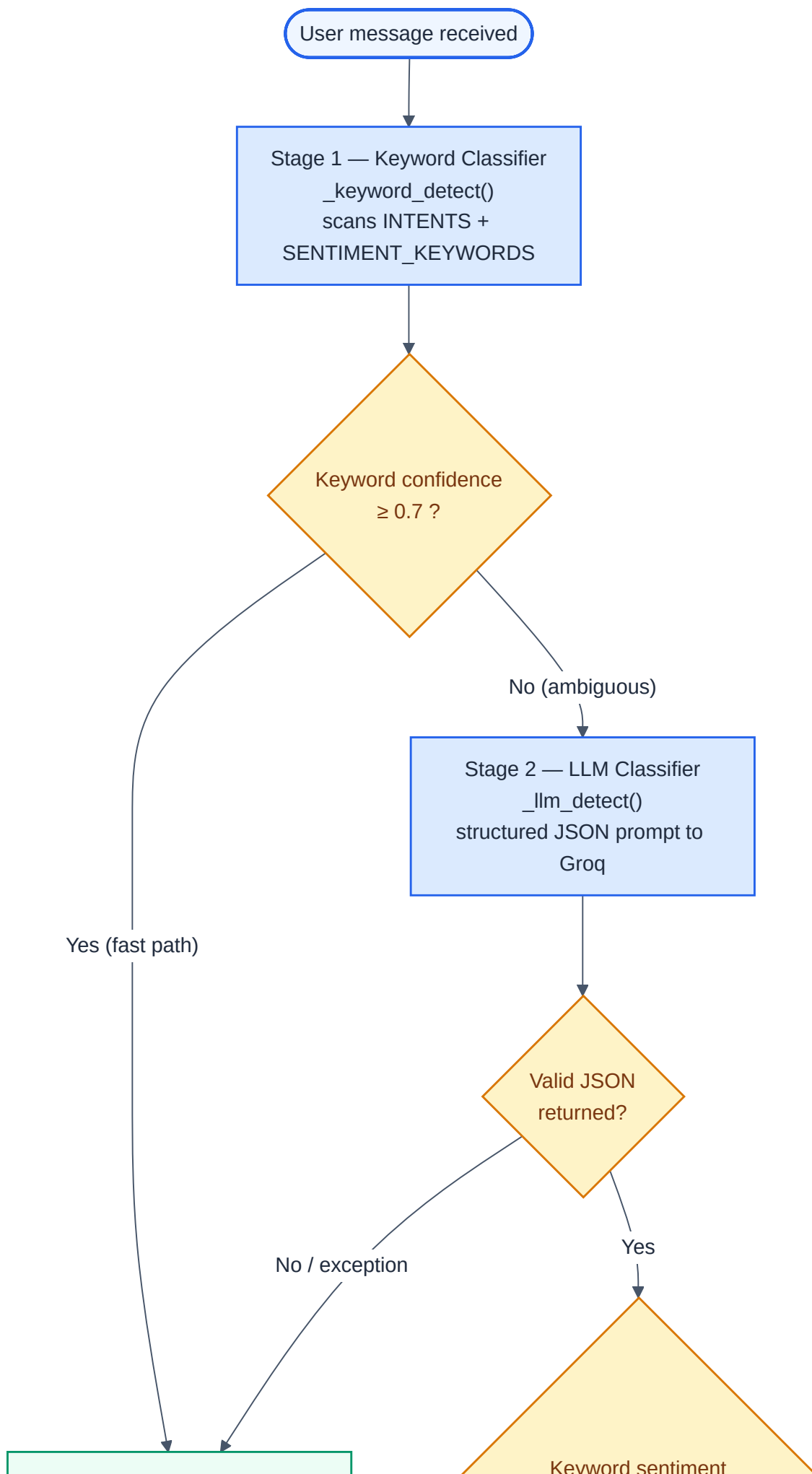
A two-stage classifier that only calls the LLM when it actually needs to

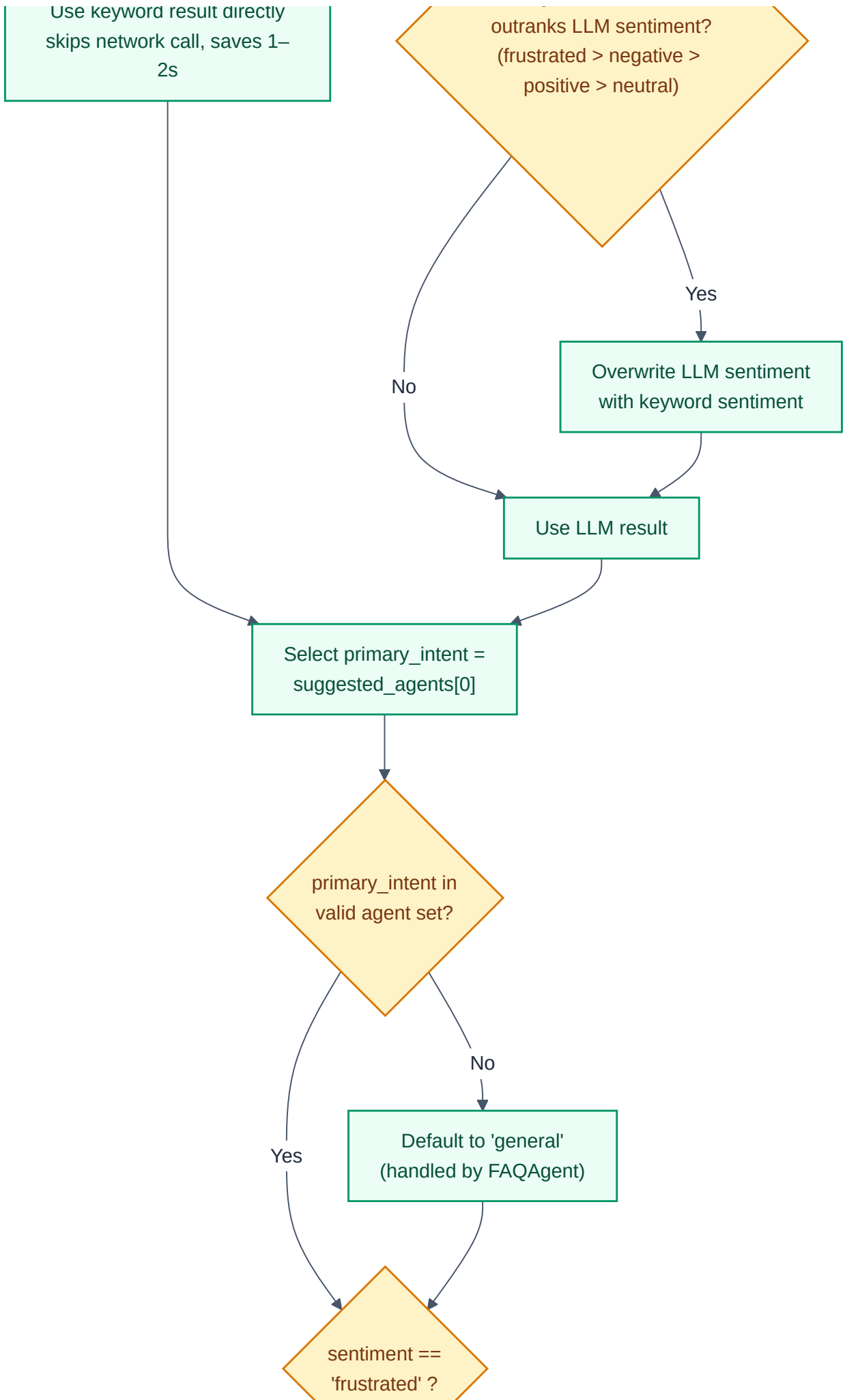
`AgentRouter.detect_intent()`, in `backend/agents/router.py`, is the single entry point that decides both *what* the customer wants (intent) and *how they feel about it* (sentiment, Chapter 12) before any agent is invoked.

11.1 Why Two Stages

Calling an LLM to classify every single message is both slower and more expensive than it needs to be — most real support messages contain unambiguous signal words ("refund," "won't turn on," "cancel my subscription") that a simple keyword match resolves correctly on its own. The router therefore always runs a fast, local, zero-network **keyword classifier** first, and only escalates to a **structured-JSON LLM classifier** when that keyword pass reports low confidence — capturing the speed of a rule-based system for the common case while retaining LLM-level judgment for ambiguous phrasing.







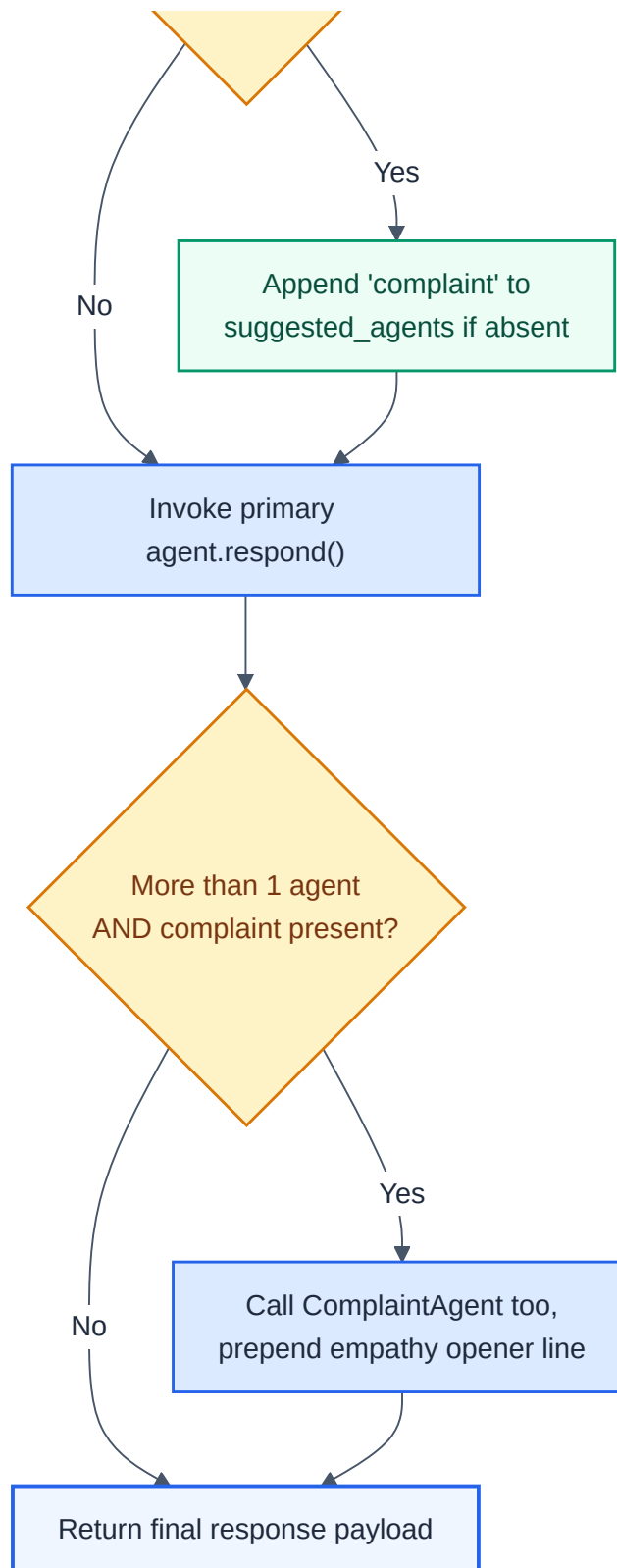


Figure 11.1 — Two-stage classification: keyword fast path, LLM escalation, sentiment-priority override, and agent dispatch.

11.2 Stage 1 — Keyword Classification

`_keyword_detect()` lower-cases the message and counts substring matches against six keyword lists totalling **158 phrases**. The intent with the most hits wins; ties are broken by dictionary order. Confidence is computed as `min(0.5 + best_score × 0.1, 0.9)` when at least one keyword matched, or a flat `0.4` when nothing matched at all (in which case the intent defaults to `general`). The top two scoring categories are

kept as `suggested_agents`, which is what later allows a message to trigger both a primary agent and a blended complaint response.

TABLE 11.1 KEYWORD-BASED INTENT CATEGORIES (BACKEND/AGENTS/ROUTER.PY)

Intent	Keywords	Representative phrases	Dispatched to
billing	35	"invoice", "techmart care", "cancel subscription"	BillingAgent
refund	21	"refund", "money back", "exchange"	BillingAgent
technical	22	"not working", "broken", "crash"	TechnicalAgent
product	45	"laptop", "specs", "compare"	ProductAgent
complaint	19	"terrible", "worst", "unacceptable"	ComplaintAgent
faq	16	"hours", "contact", "where"	FAQAgent
Total	158	—	

11.3 Stage 2 — LLM Classification

When keyword confidence is below **0.7**, `_llm_detect()` sends the message (plus recent history) to Groq with a prompt instructing the model to return only a JSON object containing `intent`, `sentiment`, and `confidence`. The LLM result is trusted only if it parses as valid JSON *and* reports its own confidence above 0.5 — otherwise the router falls back to the original keyword result rather than trusting a low-confidence or malformed LLM response. If the model omits a confidence value entirely, the parser defaults it to `0.7`.

11.4 Validation and Fallback

Whichever stage produced the final result, the router checks that `primary_intent` is a known agent domain before dispatch; an unrecognized value is coerced to `general`, which routes to `FAQAgent`. This guarantees every message receives a response from a real agent — there is no "unclassifiable, cannot answer" dead end.

A NOTE ON THE PROJECT'S OWN TEST DATA

The repository's `datasets/sample_queries.json` labels "What is your return policy?" with an expected intent of `faq`, even though the keyword list above would in fact score this string against the `refund` category first (it contains the phrase "return"). This is a genuine, minor inconsistency between that particular sample-data file's expectations and the classifier's actual keyword lists — worth knowing if that file is reused for future test-writing — and is distinct from `datasets/techmart_test_queries.json`, the file actually used to produce the accuracy figures reported in Chapter 19.



Sentiment is classified in the same `_keyword_detect()` / `_llm_detect()` pass described in Chapter 11, but is treated as a distinct signal from intent — a message can be, for example, `technical` intent with `frustrated` sentiment simultaneously, and both labels flow independently into downstream behavior.

12.1 A Four-Level Model

TABLE 12.1 SENTIMENT CATEGORIES, KEYWORD COVERAGE, AND NUMERIC SCORE

Sentiment	Keywords	Score	Example trigger words
frustrated	25	-1.0	"furious", "fed up", "ridiculous"
negative	29	-0.5	"bad", "poor", "disappointed"
positive	26	+0.8	"thanks", "great", "appreciate"
neutral	— (default)	0.0	no sentiment keywords matched

12.2 Priority Resolution

Keyword sentiment is checked in a fixed priority order — **frustrated**, then **negative**, then **positive**, then **neutral** — rather than by raw keyword count, because a genuinely frustrated message often also contains isolated positive or negative words (the code comment gives the example *"this is ridiculous, I've been waiting and no response, thanks for nothing"*, which contains "thanks" but is clearly not a positive message). This same priority order is used to decide whether a keyword-derived sentiment should override an LLM-derived one: if keyword detection found `frustrated` but the LLM stage reported `neutral`, the router keeps `frustrated`.

12.3 Downstream Consequences

Sentiment is not just logged — it actively changes what happens next in the same request:

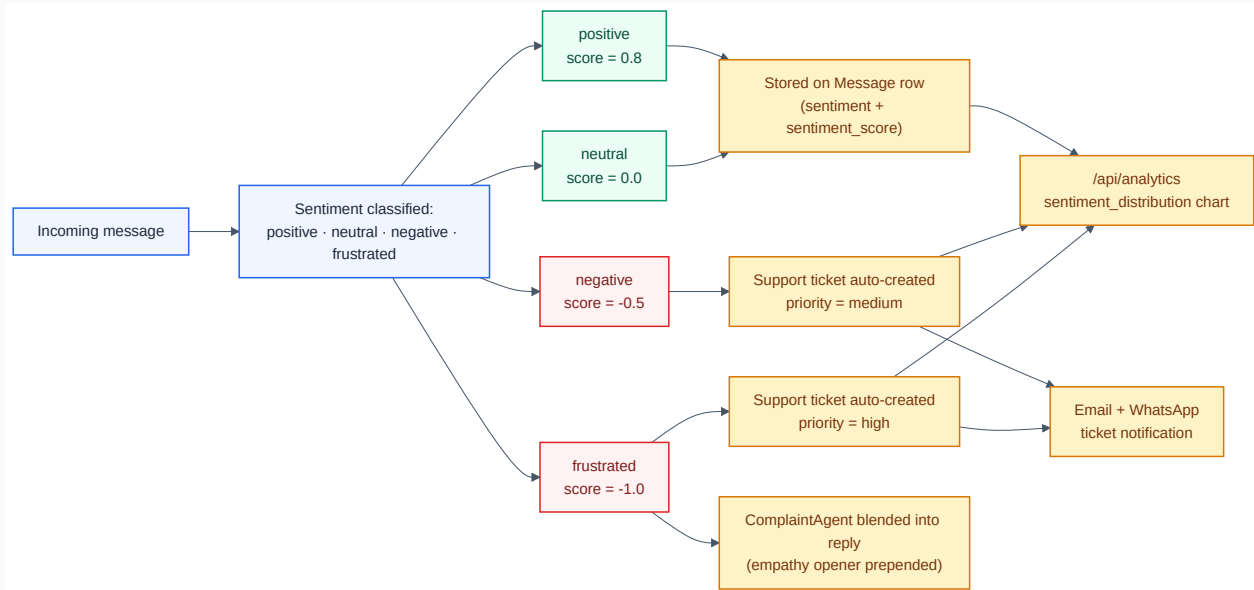


Figure 12.1 — What each sentiment level triggers: storage, ticketing, agent blending, and analytics.

- **Ticket creation.** `negative` or `frustrated` sentiment (or a `complaint` intent) auto-creates a `SupportTicket` row within the same request — `frustrated` is filed at *high* priority, `negative` at *medium*.
- **Notification.** Ticket creation triggers a best-effort email, and — if the customer supplied a phone number at registration — a WhatsApp message (Chapter 16).
- **Complaint blending.** `frustrated` sentiment causes `ComplaintAgent` to be blended into the response even when a different agent is primary (Chapter 10.3).
- **Analytics.** Every message's sentiment and score are persisted and aggregated into the `sentiment_distribution` series exposed by `GET /api/analytics` and rendered in the frontend's `AnalyticsPanel` (Chapter 14).

💡 DESIGN RATIONALE

Coupling sentiment directly to ticketing means the system does not depend on a human noticing a frustrated customer in real time — the escalation path fires automatically, synchronously, in the same request that generated the reply. Chapter 21 discusses the trade-off this synchronous design implies for request latency.

13



Backend Design

FastAPI application structure, authentication, and the relational data model

13.1 Application Bootstrap

`backend/main.py` constructs the FastAPI app and wires a `lifespan` async context manager that runs exactly once at process start: it calls `create_tables()` to ensure the schema exists, then loads the FAISS retriever from disk. `CORSMiddleware` is configured with an explicit origin allow-list — the two local dev ports (`3000` for Next.js, `5173` for Vite) on both `localhost` and `127.0.0.1`, plus the two production origins, `techmart-ai-support.vercel.app` and the backend's own `techmart-backend-j10y.onrender.com` — with credentials, all methods, and all headers permitted.

13.2 Configuration Management

All tunable behavior is centralized in a single `Settings` object in `config.py`, with every field reading from an environment variable and falling back to a sensible development default (`os.getenv("X", default)`). This is what allows the exact same codebase to run against SQLite locally and PostgreSQL in production (Chapter 18) purely through environment configuration, with no code changes.

TABLE 13.1 KEY CONFIGURATION DEFAULTS (BACKEND/CONFIG.PY)

Setting	Default	Purpose
DATABASE_URL	sqlite:///../customer_support.db	ORM connection string
GROQ_MODEL	llama-3.1-8b-instant	Chat-completion model
EMBEDDING_MODEL	all-MiniLM-L6-v2	Sentence-embedding model
CHUNK_SIZE / CHUNK_OVERLAP	600 / 80	KB chunking window
TOP_K_RESULTS	4	Chunks retrieved per query
MAX_TOKENS / TEMPERATURE	600 / 0.7	LLM generation parameters
ACCESS_TOKEN_EXPIRE_MINUTES	1440 (24h)	JWT lifetime

13.3 Authentication

`api/auth.py` implements password hashing with `passlib`'s `CryptContext` (bcrypt scheme) and stateless authentication with `python-jose`, issuing HS256-signed JWTs whose payload carries the user's ID as `sub` and a 24-hour expiry. `get_current_user()` is registered as a FastAPI dependency — via `HTTPBearer` — on every protected route, so token verification happens declaratively before a route handler body ever executes, rather than as a manual check inside each function.

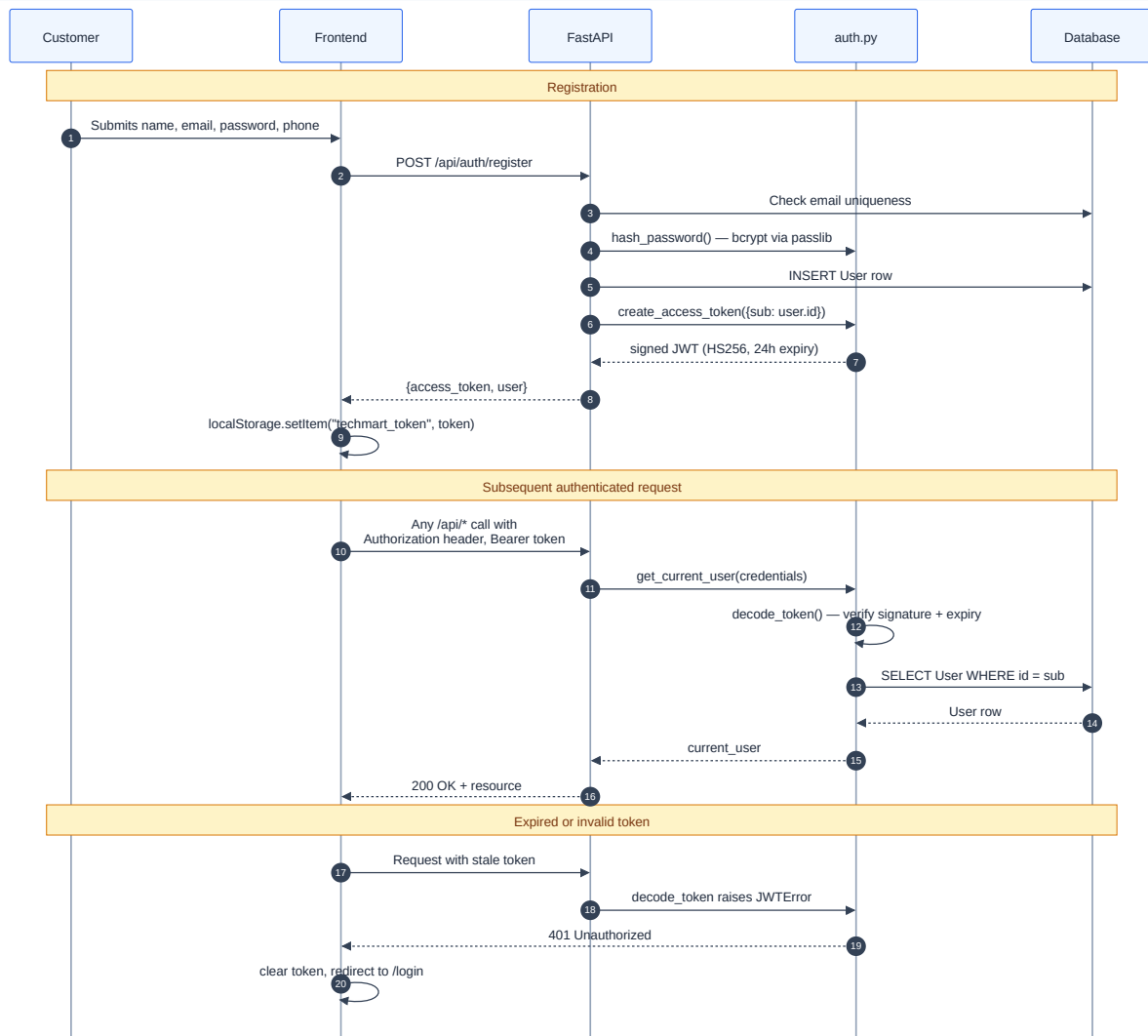


Figure 13.1 — Registration, token issuance, and per-request verification.

13.4 Rate Limiting

`check_rate_limit()` in `api/routes.py` enforces a cap of **20 messages per rolling 60-second window per user**, tracked in a plain in-process `dict` of timestamp lists keyed by user ID — no Redis or external store. This is simple and effective for a single-process deployment but, as noted in Chapter 21, does not share state across multiple backend instances.

13.5 Data Model

The relational schema (SQLAlchemy 2.0 ORM, `database/db.py`) has six tables: `User`, `ChatSession`, `Message`, `Feedback`, `SupportTicket`, and `KnowledgeBaseDoc`. Every primary key is a string UUID rather than an auto-increment integer, and every user-owned table carries a foreign key back to `User.id` so ownership can be checked on every request.

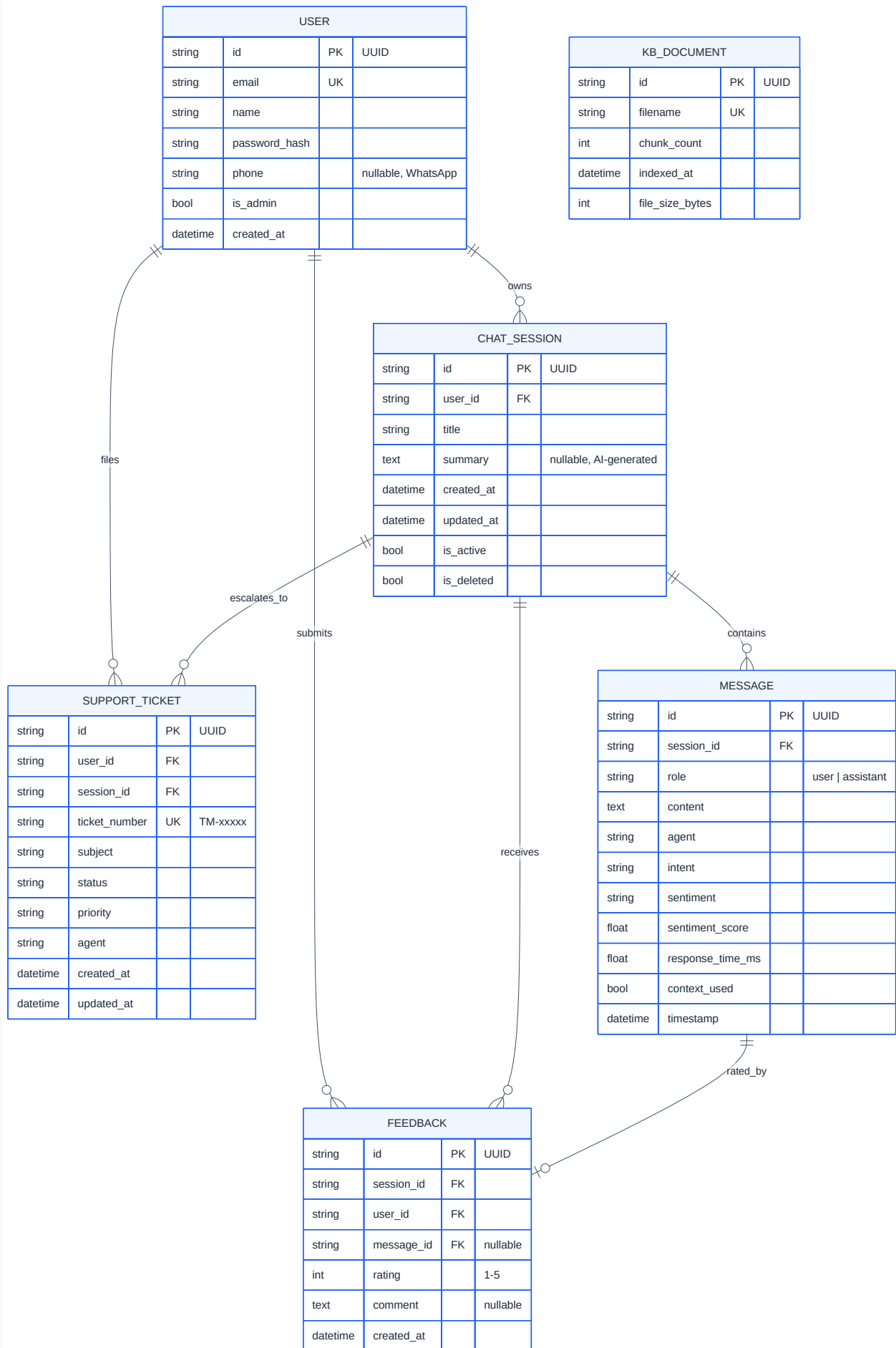


Figure 13.2 — Six-table relational schema and its foreign-key relationships.

13.6 Integration Services

`api/email_service.py` and `api/whatsapp_service.py` wrap the two outbound notification channels used for ticket and escalation alerts. Both are designed to fail soft: a misconfigured or unreachable provider logs a warning and returns `sent: false` rather than raising an exception that would break the chat response itself. Full behavior is documented in Chapter 16.



14.1 Page Structure

The frontend (Next.js 14, Pages Router) has exactly four routes: `index.js` (a pure client-side redirect to `/chat` or `/login` based on token presence), `login.js`, `register.js`, and `chat.js`. `_app.js` is the shared shell that imports `styles/globals.css` once for the whole application. There is no server-side rendering of dynamic data — every page is a client component that fetches through `services/api.js` after mount.

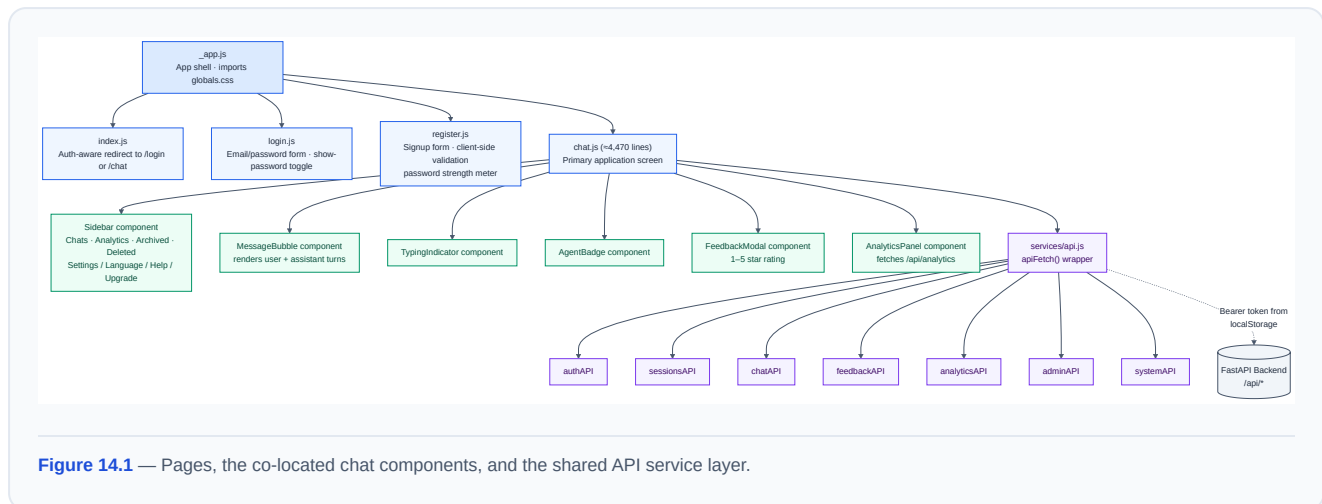


Figure 14.1 — Pages, the co-located chat components, and the shared API service layer.

14.2 `chat.js` at Scale

At approximately **4,470 lines**, `chat.js` is a deliberately monolithic single-file application screen rather than a componentized directory. It defines the exported `ChatPage` component plus six co-located, unexported components used only within this file: `Sidebar` (the largest, at roughly 2,600 lines), `MessageBubble`, `TypingIndicator`, `AgentBadge`, `FeedbackModal`, and `AnalyticsPanel`. `ChatPage` itself owns roughly fifteen `useState` hooks covering the message list, session list, active session, sidebar collapse state, dark mode, font size, voice-input listening state, and attached files.

14.3 The Sidebar

The sidebar exposes four primary views — **Chats**, **Analytics**, **Archived**, and **Deleted** — plus four expandable sections: **Settings**, **Language**, **Help**, and **Upgrade**. Several Settings items (Account & Profile, Notifications, Privacy & Security, Change Password) are implemented as static `window.alert()` informational dialogs rather than full dedicated screens — a reasonable scope decision for a project of this size, and one this report notes plainly rather than describing those items as fully interactive settings pages.

14.4 Theming and Dark Mode

The visual system is built on **CSS custom properties** layered under Tailwind utility classes, using a **blue accent palette** (`#2563EB` / Tailwind `blue-500` — `700`) rather than Tailwind's indigo scale. Dark mode is implemented as a body-level `.dark-mode` class toggle — not Tailwind's built-in `dark:` variant and not the

`prefers-color-scheme` media query — with explicit override rules for layout, sidebar, message bubbles, cards, and even raw Tailwind utility classes like `bg-white` and `text-slate-600`. These overrides account for roughly a third of the entire `globals.css` file (≈470 of 1,303 lines). The preference persists to `localStorage` under the key `techmart_dark_mode`. Font size is independently adjustable between 10px and 20px via increment/decrement controls near the message input.

14.5 Input Modalities

Beyond typed text, the composer supports **voice input** via the browser's native `webkitSpeechRecognition` API (single-shot, non-continuous, locked to `en-US` regardless of the conversation's detected language) and **file attachment** (images, PDF, plain text, and CSV, capped at 10MB) with best-effort client-side content extraction. Full behavioral detail, including a known limitation of the PDF path, is in Chapter 16.

14.6 API Service Layer

`services/api.js` centralizes network access behind a single `apiFetch()` wrapper that reads the JWT from `localStorage`, attaches it as a Bearer header, applies a 120-second client-side timeout via `AbortController`, and normalizes error handling across seven grouped API objects (`authAPI`, `sessionsAPI`, `chatAPI`, `feedbackAPI`, `analyticsAPI`, `adminAPI`, `systemAPI`). A small number of calls — notably escalation — bypass this wrapper and call `fetch()` directly inside `chat.js`, an inconsistency worth knowing about if the service layer is refactored later.

■ STACK MINIMALISM WAS A DELIBERATE CHOICE

`package.json` lists only `next`, `react`, `react-dom`, and the Tailwind toolchain — no state-management library, no UI kit, no icon package, no HTTP client library. Every icon in the interface is a hand-written inline SVG, and every network call uses the browser's native `fetch`. This keeps the dependency surface, and therefore the install size and attack surface, deliberately small.



The repository root holds four top-level directories — `backend/`, `frontend/`, `knowledge_base/`, and `datasets/` — plus four configuration files at the root itself.

15.1 Repository Root

```
multi-agent-ai-customer-support-assistant/
├── backend/           # FastAPI application (Chapter 13)
├── frontend/         # Next.js application (Chapter 14)
├── knowledge_base/   # 8 source docs, .txt + .pdf mirrors (Chapter 8)
├── datasets/         # Evaluation & sample data (Chapter 19)
├── requirements.txt  # Python dependencies
├── setup.py          # Standalone, low-RAM index bootstrap script
├── render.yaml       # Render deployment blueprint (Chapter 18)
└── .gitignore
```

15.2 Backend

```
backend/
├── main.py           # App factory, lifespan startup, CORS
├── config.py         # Settings (env-driven)
├── agents/
│   ├── base.py       # BaseAgent – shared respond() pipeline
│   ├── agents.py     # Billing / Technical / Product / Complaint / FAQ
│   ├── router.py     # AgentRouter – intent + sentiment classification
│   └── llm_client.py  # Groq wrapper – retry, backoff, fallback
├── api/
│   ├── routes.py     # 31 endpoint handlers
│   ├── auth.py       # JWT + bcrypt
│   ├── email_service.py # SendGrid / SMTP notifications
│   └── whatsapp_service.py # Twilio WhatsApp sandbox
├── rag/
│   ├── document_processor.py # Load + chunk knowledge base
│   ├── embeddings.py   # EmbeddingManager (MiniLM)
│   └── retriever.py    # FAISSRetriever – build / load / search
├── database/
│   └── db.py          # SQLAlchemy models + session factory
├── models/
│   └── schemas.py     # Pydantic request/response schemas
└── vectorstore/faiss_index/
    ├── faiss.index    # 217×384 IndexFlatIP (bundled)
    └── chunks.pkl     # Chunk metadata, dict-format
```

15.3 Frontend

```
frontend/
├── pages/
│   ├── _app.js       # Shared shell, imports globals.css
│   ├── index.js      # Auth-aware redirect
│   ├── login.js      # Login form
│   ├── register.js   # Signup form
│   └── chat.js        # ~4,470 lines – the application itself
├── services/
│   └── api.js         # apiFetch() + 7 grouped API objects
├── styles/
│   └── globals.css    # 1,303 lines – incl. ~470 dark-mode overrides
```

```
├─ next.config.js
├─ tailwind.config.js
└─ package.json # next, react, react-dom + Tailwind toolchain only
```

15.4 Knowledge Base and Datasets

`knowledge_base/` holds eight `.txt` files (the ones actually ingested — Chapter 8) alongside eight matching `.pdf` mirrors intended for human reading rather than ingestion. `datasets/` holds five JSON files used for manual and exploratory testing (`sample_queries.json` , `techmart_test_queries.json` , `evaluation_results.json` , `sample_users.json`) plus two CSV files, `banking77_train.csv` and `banking77_test.csv` — the public Banking77 intent-classification corpus (over 13,000 labeled banking queries). A full-repository search confirms this corpus is not currently referenced by any Python or JavaScript source file; it is present as reference material, not wired into training or evaluation code.

■ WHAT'S DELIBERATELY ABSENT

There is no `README.md` , no `.env.example` , and no automated test directory in the delivered project. This report is written to stand in for the first of those — a complete, accurate technical reference — and Chapter 19 addresses the absence of the third honestly rather than describing tests that do not exist.



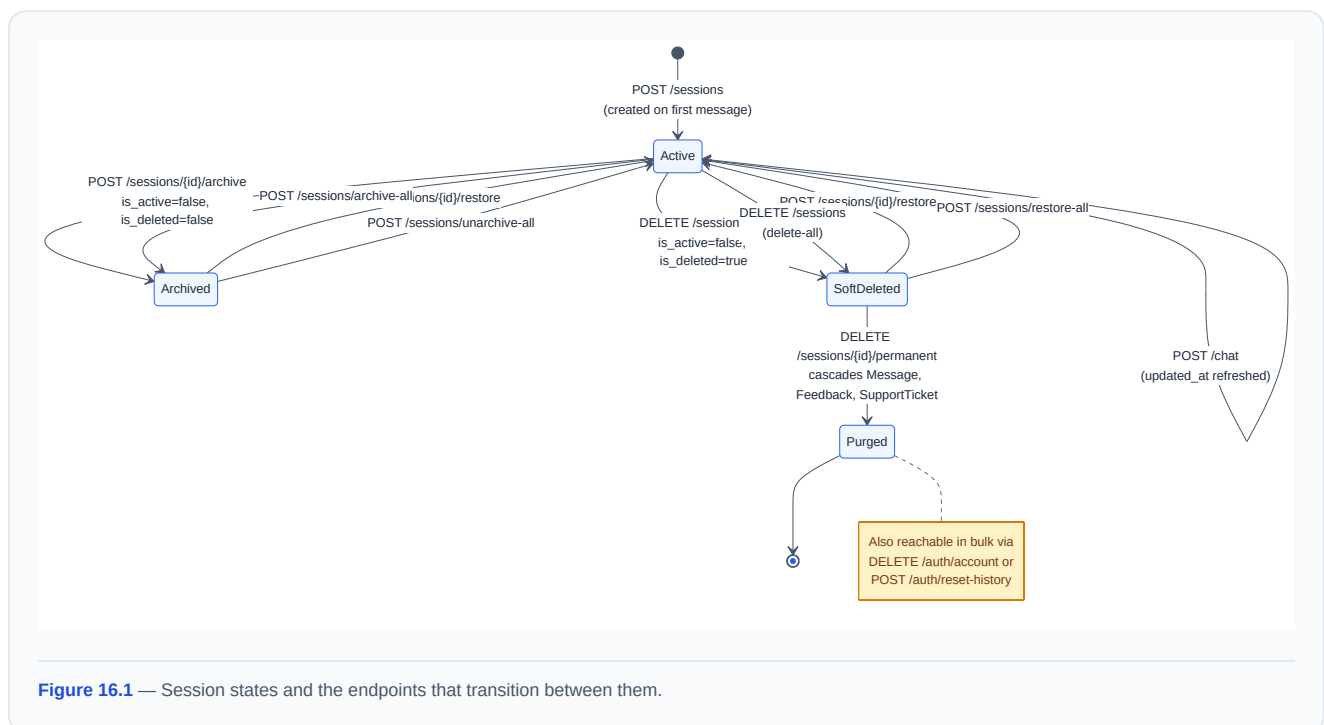
This chapter goes one level deeper than Chapters 13–14 into specific pieces of logic that are individually small but collectively define the system's day-to-day behavior.

16.1 LLM Client Resilience

`LLMClient.chat()` retries a failed Groq call up to **three times**, waiting **2 seconds** after the first failure and **4 seconds** after the second ($(\text{attempt} + 1) \times 2$) before giving up and returning a canned, agent-appropriate fallback string rather than propagating an exception to the customer. This logic was added specifically to eliminate intermittent "Failed to fetch" -style timeout errors observed against the LLM provider during development, and it is the reason the chat endpoint essentially never returns a hard 500 error for LLM-side failures.

16.2 Session Lifecycle

A `ChatSession` moves between three logical states — active, archived, and soft-deleted — via `is_active` and `is_deleted` boolean flags, with a fourth, terminal state (hard-deleted) reachable only through an explicit permanent-delete call that cascades to the session's messages, feedback, and tickets.



Both archiving and soft-deletion support single-session and bulk (`-all`) variants, and both are individually and fully reversible via restore endpoints — nothing is unrecoverable until a permanent-delete call is made. Session titles are auto-generated from the first user message rather than left as "New Chat," and the sidebar list re-fetches on every session mutation so the title updates without a manual page reload.

16.3 Ticketing and Escalation

Automatically created support tickets receive an identifier of the form `TM-XXXXX`, where the five digits are drawn with `random.randint(10000, 99999)` — a randomly sampled, not sequentially incremented, number. Manual escalations (triggered by the frontend's "Escalate to a human agent" button, after a confirmation dialog) receive a different identifier scheme, `ESC-` followed by the first eight characters of the session's UUID, upper-cased.

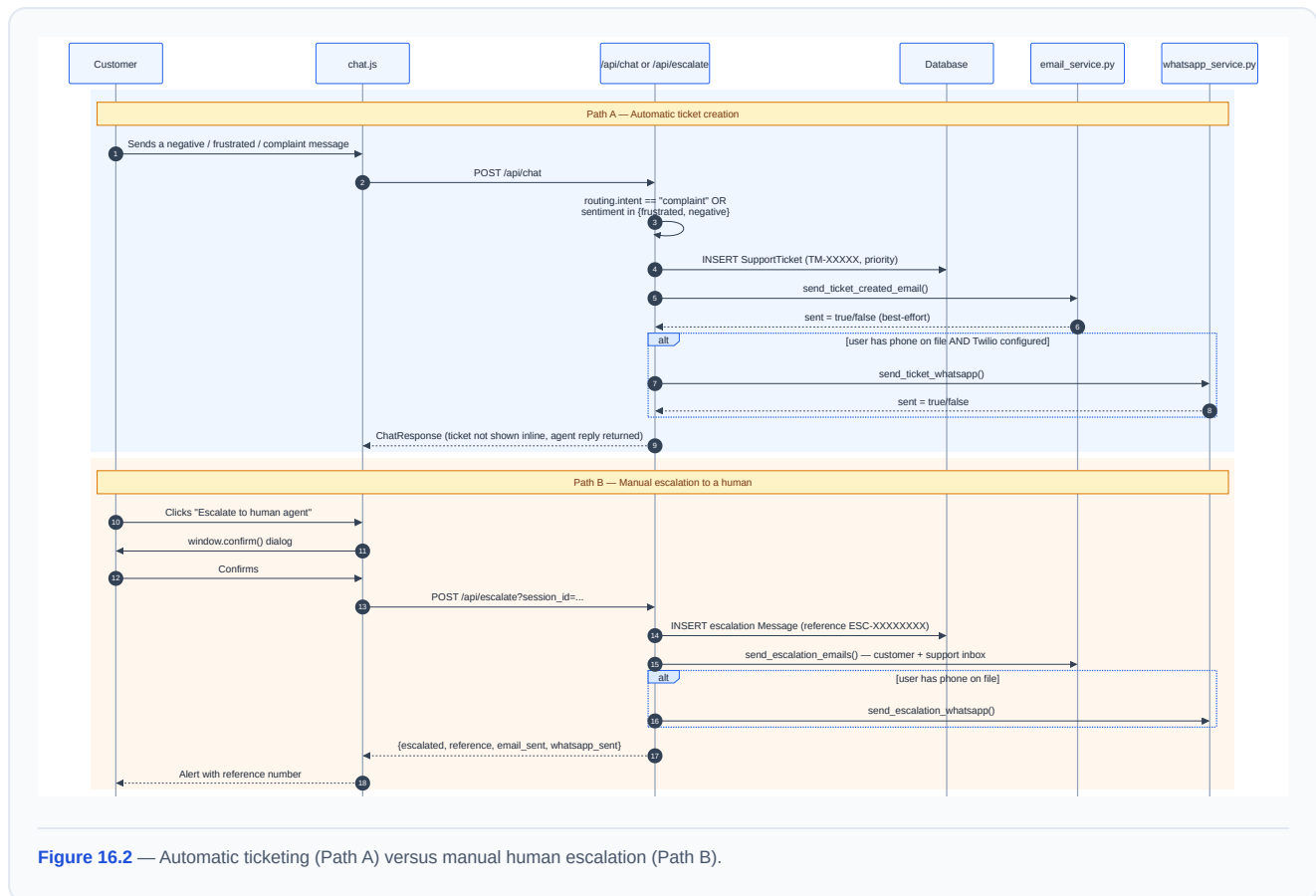


Figure 16.2 — Automatic ticketing (Path A) versus manual human escalation (Path B).

16.4 Notification Channels

`email_service.py` prefers the SendGrid HTTP API when `SENDGRID_API_KEY` is configured, and falls back to SMTP (compatible with Gmail's SMTP relay) otherwise — both paths converge on the same templated ticket/escalation messages. `whatsapp_service.py` sends through the Twilio WhatsApp *sandbox* API, and is only invoked when the customer has supplied a phone number at registration (the field is explicitly labeled "optional — for WhatsApp alerts" on the sign-up form). Both services are queried for their live configuration status through `GET /api/email/status` and `GET /api/whatsapp/status`.

16.5 Multilingual Response Generation

Language adaptation is heuristic, not a call to a dedicated language-ID model:

`BaseAgent._detect_language()` checks the message for Unicode code-points specific to Devanagari (Hindi), Hiragana/Katakana (Japanese), Arabic script, and CJK ideographs (Chinese), and falls back to common-word matching for Spanish, French, and German, defaulting to English if nothing matches. The detected language name is injected into the system prompt with an explicit instruction to reply only in that language. This is a genuinely functional, automatic per-message capability — distinct from the chat

sidebar's manual language-selector menu, whose non-English entries currently display an informational "coming soon" message rather than changing any request behavior (the underlying `selectedLanguage` state is set once at component initialization and not otherwise read or written).

16.6 Voice Input and File Attachments

Voice input uses the browser-native `webkitSpeechRecognition` API in single-shot mode, hard-coded to `en-us`. File attachment accepts images (JPEG/PNG/GIF/WebP), PDF, plain text, and CSV up to 10MB; images are given a local blob preview and represented to the backend only as a `[Image file: name]` placeholder tag (the LLM never receives actual image pixel data, since no vision-model call is made), plain text and CSV are read directly via `FileReader.readAsText()`, and PDF is *also* read via `readAsText()` rather than a dedicated PDF-parsing library — meaning extracted PDF content is best-effort and will often include binary artifacts, since `readAsText()` is not a PDF parser. `PyPDF2` is already listed in `requirements.txt` and could parse PDFs properly on the backend if this path is revisited (Chapter 22).

16.7 Chat Export

The "Export" action in the chat header requires no backend call at all — it serializes the current session's messages into a plain-text transcript client-side, wraps it in a `Blob`, and triggers a browser download named after the session title.

16.8 Default Administrative Account

`setup.py` seeds a default administrator account (`admin@gmail.com`) during first-time local setup, and the credential is intentionally surfaced on the login screen itself as a labeled "Demo" hint for evaluators. This is a reasonable convenience for an academic/portfolio deployment but is flagged as a pre-production action item in Chapter 25, Security Considerations.



Every route below is defined in `backend/api/routes.py` and mounted under the `/api` prefix. Unless noted otherwise, every endpoint requires a valid `Authorization: Bearer <token>` header (Chapter 13.3) and returns JSON.

17.1 Authentication

Method & Path	Auth	Purpose
POST /auth/register	No	Create a user; returns a JWT immediately (auto-login on signup)
POST /auth/login	No	Verify credentials; returns a JWT
GET /auth/me	Yes	Return the authenticated user's own profile
DELETE /auth/account	Yes	Permanently delete the account and all owned data
POST /auth/reset-history	Yes	Bulk-delete all of the user's sessions/messages, keeping the account

17.2 Chat & Sessions

Method & Path	Purpose
POST /chat	Send a message; returns the agent's reply plus routing metadata
GET /sessions	List the user's active sessions
POST /sessions	Explicitly create a new empty session
GET /sessions/{id}/history	Full message history for one session
GET /sessions/{id}/summary	AI-generated summary of a session
DELETE /sessions/{id}	Soft-delete one session
DELETE /sessions	Soft-delete all of the user's sessions
POST /sessions/{id}/archive	Archive one session
POST /sessions/archive-all	Archive every active session
GET /sessions/archived	List archived sessions
GET /sessions/deleted	List soft-deleted sessions
POST /sessions/{id}/restore	Restore one archived or deleted session
POST /sessions/unarchive-all	Restore every archived session
POST /sessions/restore-all	Restore every soft-deleted session
DELETE /sessions/{id}/permanent	Hard-delete a session and cascade its messages/feedback/tickets

17.3 Feedback, Analytics, Tickets & Escalation

Method & Path	Purpose
POST /feedback	Submit a 1–5 star rating and optional comment for a session
GET /analytics	Aggregated stats: intent/agent/sentiment distributions, volumes
POST /tickets/create	Manually create a support ticket
GET /tickets	List the user's tickets
POST /escalate	Escalate the current session to a human agent

17.4 Knowledge Base Admin, Notifications & System

Method & Path	Purpose
GET /admin/knowledge-base	Per-document chunk counts and index stats
POST /admin/knowledge-base/rebuild	Re-run ingestion and hot-swap the FAISS index
POST /admin/knowledge-base/upload	Add a new document and trigger a rebuild
GET /email/status	Whether SendGrid/SMTP is currently configured
GET /whatsapp/status	Whether Twilio is currently configured
GET/HEAD /health	Liveness probe; reports whether the FAISS index is loaded (no auth required)

17.5 Representative Request / Response — POST /api/chat

```
// Request
{
  "message": "My laptop won't turn on",
  "session_id": "3f9a1c2e-..." // omit to start a new session
}

// Response — 200 OK
{
  "session_id": "3f9a1c2e-...",
  "message_id": "7b2d...",
  "response": "Let's troubleshoot this together. First, ...",
  "agent": "technical",
  "intent": "technical",
  "sentiment": "neutral",
  "sentiment_score": 0.0,
  "response_time_ms": 1180.4,
  "context_retrieved": true,
  "timestamp": "2026-07-19T10:22:41Z"
}
```

17.6 Error Handling

Authentication failures return **401** (invalid credentials at login, or a missing/expired/tampered JWT on any protected route). Exceeding 20 messages within 60 seconds returns **429**. Accessing another user's session or ticket returns **404** rather than 403, so as not to confirm the resource's existence to an unauthorized caller. Malformed request bodies are rejected with **422** by FastAPI's automatic Pydantic validation before the route handler runs at all. Unhandled exceptions inside the chat pipeline itself are the one category the system tries hardest to avoid ever surfacing as a 500 — the LLM retry/fallback logic in

Chapter 16.1 exists specifically to keep `POST /chat` returning a 200 with a graceful fallback message rather than an error.

18



Deployment

Two managed platforms, environment-driven configuration, and a documented production tuning pass

18.1 Topology

The frontend deploys to **Vercel** as a Next.js production build; the backend deploys to **Render** as a persistent Python web service defined declaratively in `render.yaml`. There is no containerization (no `Dockerfile` in the repository) — Render's native Python buildpack installs dependencies directly from `requirements.txt`.

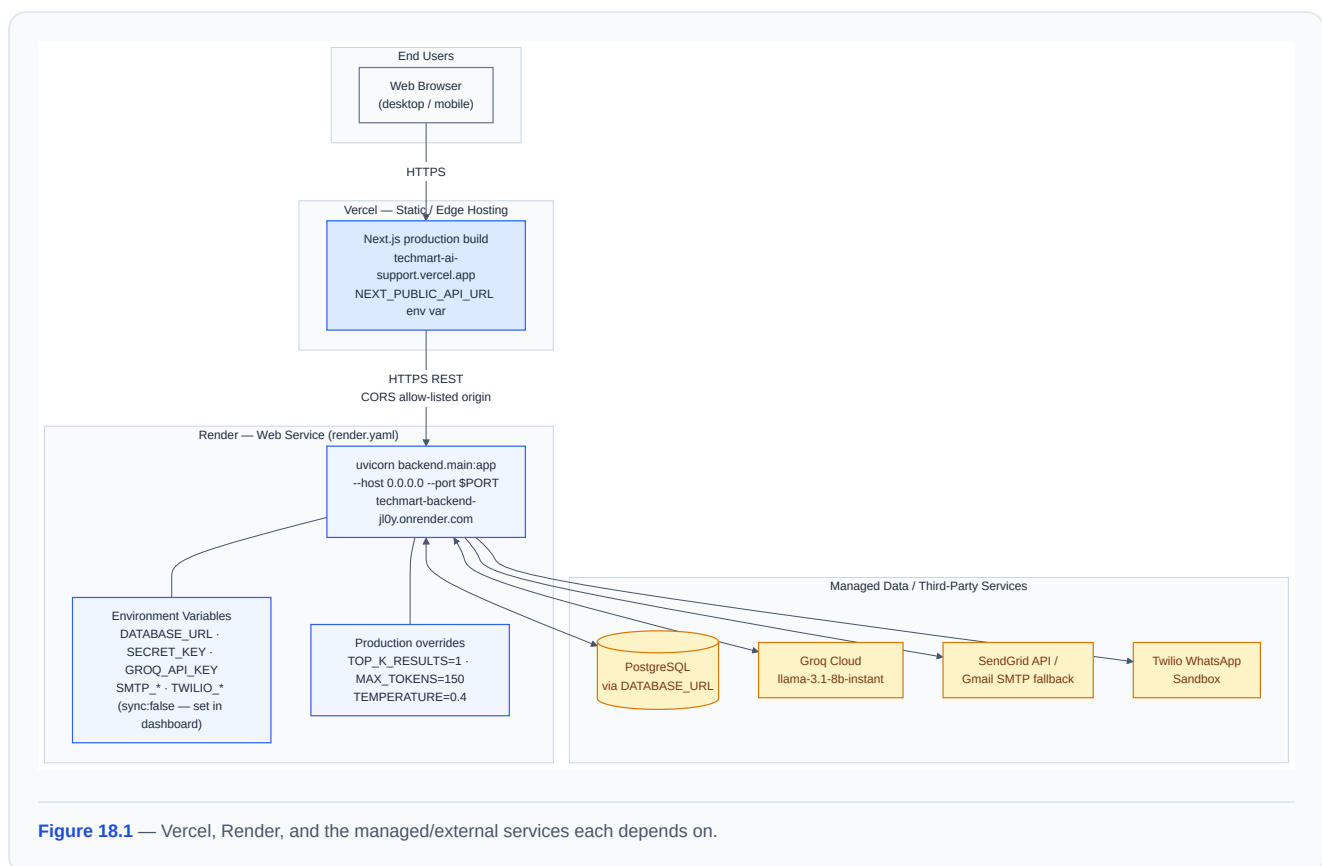


Figure 18.1 — Vercel, Render, and the managed/external services each depends on.

18.2 Backend Service Definition

`render.yaml` declares a single `web` service, `techmart-backend`, built with `pip install -r requirements.txt` and started with `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`. Secrets — `DATABASE_URL`, `SECRET_KEY`, `GROQ_API_KEY`, SMTP and Twilio credentials — are all marked `sync: false`, meaning Render will prompt for them in its dashboard rather than storing them in the blueprint file itself; non-secret configuration (model names, SMTP host, tuning parameters) is set directly as literal values in the blueprint.

18.3 Frontend Configuration

The only environment-specific frontend setting is `NEXT_PUBLIC_API_URL` (`next.config.js`), which points the deployed app at the Render backend URL and defaults to `http://localhost:8000/api` for local development.

18.4 Production Tuning

Three retrieval/generation parameters are deliberately overridden in production, trading some answer richness for latency headroom on Render's infrastructure:

TABLE 18.1 DEVELOPMENT DEFAULTS VS. PRODUCTION OVERRIDES

Parameter	Local default	Production (render.yaml)	Effect
TOP_K_RESULTS	4	1	Fewer chunks retrieved → shorter prompt → faster generation
MAX_TOKENS	600	150	Shorter generated replies → lower latency
TEMPERATURE	0.7	0.4	More deterministic, less exploratory phrasing

18.5 Cold Starts

Render's free/hobby web-service tier spins the process down after a period of inactivity and takes on the order of a minute to wake back up on the next request. The frontend accounts for this directly: `chat.js`'s network error handling includes a specific user-facing message — *"Connection error. Backend may be waking up — wait 60 seconds and retry."* — rather than a generic failure notice, so the delay is explained rather than left to look like a bug.

⚠ WHAT ISN'T HERE

There is no CI/CD pipeline definition (no GitHub Actions workflow) in the repository — deployment is Render's and Vercel's native git-push-to-deploy behavior. There is also no separate staging environment; the `render.yaml` defines exactly one backend service.



This chapter reports only what the repository's own test artifacts demonstrate. There is no automated test suite — no `pytest`, `unittest`, or frontend test files ship in the project — so "testing" here means a recorded, reproducible manual evaluation run, not continuous automated coverage.

19.1 The Evaluation Benchmark

`datasets/techmart_test_queries.json` defines ten hand-written queries spanning all five agent domains and three languages (English, Hindi, Spanish), each with an expected intent and expected agent.

`datasets/evaluation_results.json` records the outcome of actually running that benchmark against the deployed classifier — the specific model (`llama-3.1-8b-instant`), embedding model, and index size are logged alongside the results, not just the pass/fail count.

90%

INTENT ACCURACY

90%

AGENT ROUTING ACCURACY

~1.2s

AVG. RESPONSE TIME

19.2 Functional and Integration Coverage

Beyond classification accuracy, the recorded evaluation implicitly exercises the full integration path for each of the ten queries — retrieval, prompt assembly, and live LLM generation — since response time is measured per query (individual times in the recorded run span roughly 0.8–1.8 seconds), not simulated. This constitutes an integration test of the request lifecycle described in Chapter 7, even though it is not packaged as an automated regression suite.

19.3 The One Miss

Of the ten benchmark queries, nine were classified correctly on both axes. The single miss — a message combining a billing statement ("I paid yesterday") with a technical symptom ("Premium is still locked") — was expected to route to `billing` but was classified as `general`. The benchmark file's own annotation for this query anticipates the difficulty, noting it "should also invoke technical agent" — i.e., it is a compound-intent message that a single-label classifier is structurally not designed to split across two agents. This is a genuine, representative edge case rather than a cherry-picked easy failure, and is revisited as a limitation in Chapter 21.

19.4 Manual Test Fixtures

`datasets/sample_users.json` and `datasets/sample_queries.json` provide additional hand-crafted fixtures for manually exercising registration and chat flows during development; they were not run through an automated harness and are not the source of the accuracy figures above (Chapter 11.4 notes one labeling inconsistency in the latter file).

19.5 Edge Cases Observed

- **Compound intents** — a single message spanning two domains (Section 19.3) resolves to one primary agent, not a blended multi-domain answer, unless the secondary signal is specifically `frustrated` sentiment (Chapter 10.3).
- **LLM transient failures** — covered by the three-attempt retry and template fallback in Chapter 16.1, exercised in development against real intermittent provider timeouts.
- **Malformed knowledge-base encodings** — covered by the four-encoding fallback chain in Chapter 8.1.
- **Cold-start latency** — the first request after a Render free-tier sleep period is materially slower than the measured steady-state figures above (Chapter 18.5).

⚠ SAMPLE SIZE

Ten queries is a small benchmark; a 90% figure on $n=10$ means exactly nine correct classifications, and the true error rate could reasonably differ from 10% by several points in either direction on a larger, independently-labeled set. This report treats the figure as an honest, reproducible data point from the delivered project — not as a claim of statistically robust accuracy.

20



Advantages

What this specific implementation does well, grounded in the codebase, not marketing language

Answers are grounded, not guessed

Every agent response is conditioned on retrieved TechMart source text (Chapter 9), structurally reducing the chance of a confidently wrong policy statement compared to an ungrounded chat prompt.

Domain-appropriate tone by design

Five separately-tuned personas (Chapter 10) mean a complaint gets empathy and a billing question gets precision, without one prompt trying to do both.

Frustration is acted on automatically

Negative and frustrated sentiment triggers ticketing and multi-channel notification synchronously (Chapter 12) — no human has to be watching a queue.

Retrieval is fast at this scale

A 217-vector flat FAISS index returns exact nearest neighbors in low single-digit milliseconds — no approximation error, no index-build latency at query time.

Genuinely functional multilingual replies

Unicode-block and keyword-based language detection (Chapter 16.5) lets the same English knowledge base serve replies in eight languages without maintaining eight parallel prompt sets.

Resilient to LLM provider hiccups

Retry-with-backoff and a template fallback (Chapter 16.1) mean a transient Groq error degrades gracefully rather than breaking the chat.

Conversation history as a first-class citizen

Archive, soft-delete, restore, permanent-delete, and export are all implemented and reversible except the final step (Chapter 16.2) — history is never a black box.

Evaluated, not just claimed

A recorded benchmark run with per-query timing and per-query correctness (Chapter 19) is a materially stronger evidence base than an unverified accuracy claim.

20.1 Engineering Hygiene

Beyond individual features, several structural choices reflect solid engineering discipline for a project of this scope: configuration is environment-driven end to end (Chapter 13.2), the retrieval and generation layers are cleanly separated from the routing layer, every table uses UUID primary keys, and the frontend's dependency footprint is deliberately minimal (Chapter 14.6) rather than accumulating unnecessary packages.



Every item below was directly observed while producing this report and can be traced to a specific file. None are hypothetical.

21.1 Architecture and Scaling Constraints

- **Single-writer database by default.** Without a `DATABASE_URL` override, the app runs against a single SQLite file — fine for demonstration and low-concurrency use, unsuitable for concurrent multi-instance writes. PostgreSQL is fully supported (the driver ships in `requirements.txt`) but requires an explicit environment variable to activate.
- **In-process rate limiting.** The 20-messages-per-minute limiter (Chapter 13.4) is a plain Python dictionary — it resets on every process restart and does not share state across multiple backend instances, so it would under-enforce behind a multi-instance/load-balanced deployment.
- **No background job queue.** Ticket-notification emails and WhatsApp messages are sent synchronously inside the same request/response cycle as the chat reply (Chapter 16.4) — a slow SMTP or Twilio call adds directly to the customer-perceived response time rather than being offloaded.
- **Flat FAISS index does not scale indefinitely.** Exact search is ideal at 217 vectors but would need to move to an approximate index (IVF/HNSW) at a materially larger document count (Chapter 9.4 / Chapter 22).

21.2 Known Code-Level Inconsistencies

Issue	Where	Practical effect
<code>build_system_prompt()</code> never returns its result	<code>agents/agents.py</code> — <code>BillingAgent</code>	Billing replies use a generic fallback prompt rather than the intended billing-specific rules text; explicitly commented in source as unfixed
Two ingestion paths use different index types	<code>rag/retriever.py</code> (L2) vs <code>setup.py</code> (IP)	The bundled index is <code>IndexFlatIP</code> ; rebuilding via the retriever's own <code>build_index()</code> would instead produce <code>IndexFlatL2</code> , a different distance metric
Two ingestion paths use different chunk sizes	600/80 (config default) vs 300/30 (<code>setup.py</code>)	Rebuilding via <code>setup.py</code> vs. the admin rebuild endpoint would not reproduce identical chunk boundaries
Pricing figures duplicated in two places	<code>agents.py</code> hardcoded rules vs. <code>knowledge_base/pricing.txt</code>	The Billing agent's hardcoded "rules" text and the RAG-retrieved pricing document are two independent sources of truth that could drift out of sync
Escalation call bypasses the shared API client	<code>chat.js</code> — <code>direct fetch()</code>	Minor inconsistency versus the centralized <code>services/api.js</code> pattern used elsewhere

21.3 Feature Completeness Gaps

- **Sidebar language switcher is cosmetic.** Selecting a non-English language from the sidebar menu shows a "coming soon" message rather than changing behavior; the system's real multilingual

capability is the automatic, per-message backend detection (Chapter 16.5), which is independent of this control.

- **Client-side PDF parsing is naive.** Attached PDFs are read with the browser's plain-text `FileReader.readAsText()`, not a PDF-parsing library, so extracted content from a real PDF will generally include unreadable binary artifacts (Chapter 16.6).
- **Voice input is English-only** regardless of the conversation's detected language (Chapter 16.6).
- **Several sidebar "Settings" entries are static informational dialogs** rather than functional screens (Chapter 14.3).
- **Twilio integration targets the WhatsApp sandbox**, not a production-approved WhatsApp Business sender (Chapter 18.2).

21.4 Testing and Evaluation Scope

As detailed in Chapter 19, there is no automated regression test suite, and the reported 90% accuracy figures rest on a ten-query benchmark — real and reproducible, but small. The Banking77 corpus bundled in `datasets/` is not currently used anywhere in the pipeline (Chapter 15.4).

21.5 Operational and Security Notes

`setup.py` seeds a default administrator account whose credentials are also displayed on the production login page as a demo hint (Chapter 16.8) — appropriate for an evaluated academic/portfolio deployment, not for an unmodified production launch. Full treatment in Chapter 25.

WHY THIS SECTION EXISTS IN THIS FORM

The generation instructions behind this report explicitly required limitations to be reported honestly rather than omitted or softened. Every item above is a specific, verifiable characteristic of the delivered code — not a generic disclaimer — and several (Sections 21.2 and 21.3 in particular) are exactly the kind of detail a technical reviewer or hiring manager would want to see a candidate already aware of.



Every item here is scoped as an extension of what already exists — none require re-architecting the system, and none are described as already implemented.

Background task queue

Move email/WhatsApp notification dispatch (Chapter 16.4) off the request path using Celery, RQ, or FastAPI's own `BackgroundTasks`, removing notification latency from the customer-facing response time.

Distributed rate limiting

Replace the in-process dictionary (Chapter 13.4) with a Redis-backed counter so the limit holds correctly across multiple backend instances.

Unify the two ingestion code paths

Retire `setup.py`'s independent chunking/indexing logic in favor of always calling the same `rag/document_processor.py` + `retriever.build_index()` path, eliminating the chunk-size and index-type divergence noted in Chapter 21.2.

Fix and re-verify BillingAgent's prompt

Add the missing `return` in `build_system_prompt()` and re-run the Chapter 19 benchmark to confirm billing-specific responses improve.

Real PDF parsing

`PyPDF2` already ships in `requirements.txt`; routing attached PDFs through a backend extraction endpoint instead of client-side `readAsText()` would make the file-upload feature (Chapter 16.6) reliable for real documents.

Wire up the language switcher

Connect the sidebar's language menu (Chapter 21.3) to the same detection/response mechanism already implemented server-side, so manual selection and automatic detection agree.

Automated test suite

A `pytest` suite covering the router's keyword classifier, the chunking boundary logic, and the API contract would convert the current one-off benchmark (Chapter 19) into continuous regression protection.

Approximate index for larger knowledge bases

If the knowledge base grows well beyond a few hundred chunks, migrating from a flat FAISS index to IVF or HNSW would preserve low query latency at scale (Chapter 9.4).

Production WhatsApp sender

Graduate from the Twilio sandbox number to an approved WhatsApp Business sender ahead of any real customer-facing use (Chapter 18.2).

CI/CD pipeline

A GitHub Actions workflow running lint/type-checks and the future test suite on every push would close the gap noted in Chapter 18.5.



The TechMart scenario is a concrete instantiation of a pattern — chunk-embed-index a policy corpus, classify intent and sentiment, route to a specialized grounded agent — that transfers directly to any organization whose support volume splits into a handful of recurring categories, each backed by its own reference documents.

TABLE 23.1 ILLUSTRATIVE TRANSFER OF THE SAME ARCHITECTURE TO OTHER VERTICALS

Industry	Analogous agent split	Analogous knowledge base
E-commerce / retail	Billing, Shipping, Returns, Product, FAQ	Order policy, catalog, return policy documents
Banking / fintech	Account, Card, Fraud, Loan, FAQ	Terms of service, fee schedules — note the bundled Banking77 corpus (Chapter 15.4) is a direct fit for this vertical's intent-classification needs if activated
Telecom	Billing, Network Troubleshooting, Plans, Complaint, FAQ	Plan terms, coverage/outage documentation
SaaS / B2B software	Billing, Technical, Onboarding, Bug Reports, FAQ	Product docs, API reference, changelog
Internal IT helpdesk	Access/Accounts, Hardware, Software, Policy, FAQ	Internal IT policy and how-to documentation

In every case, the underlying engineering — Chapters 8 through 12 — is domain-agnostic: swapping the eight `.txt` files and the five agent personas for a different vertical's documents and categories requires no architectural change, only new knowledge-base content and updated `INTENTS`/agent definitions.

■ SCOPE OF THIS CLAIM

This chapter describes architectural transferability, not a claim that the delivered project has been deployed in any of these industries. TechMart Electronics remains the only implemented and evaluated domain.



24.1 Customer Experience

A ~1.2-second average response time (Chapter 19.1) is fast enough to sustain a real-time chat experience, and grounding every answer in retrieved TechMart documentation (Chapter 9) directly targets the specific trust failure — confident, ungrounded answers — that motivated this project (Chapter 2.1).

24.2 Automation and Triage

Five specialized agents perform the first-line triage a human support team would otherwise do manually, and automatic ticket creation on complaint/negative-sentiment messages (Chapter 12.3) means every message that plausibly needs human follow-up is already logged, prioritized, and — where a phone number is on file — actively surfaced to the customer via two channels, without requiring a support agent to be monitoring a live queue.

24.3 Cost Structure

The inference cost profile is deliberately lightweight: an 8-billion-parameter model accessed through Groq's low-latency API, a CPU-friendly 22-million-parameter embedding model, and a zero-training-cost flat vector index. Combined with free/hobby-tier hosting on Vercel and Render, the entire system can run at near-zero fixed infrastructure cost at low volume — the honest trade-off, documented in Chapter 18.5, is cold-start latency on the free hosting tier rather than dollar cost.

24.4 Response Quality and Consistency

Because every agent draws from the same eight source documents rather than improvising, answers to the same underlying policy question are far more likely to be consistent across conversations than they would be from an ungrounded model or from inconsistently-trained human agents.

24.5 Scalability — Honestly Scoped

The application layer itself is stateless per request and could in principle run behind a load balancer across multiple instances; what currently would *not* scale correctly out of the box are the in-process rate limiter and the default SQLite database (Chapter 21.1), both of which have a documented, realistic path to a distributed-safe configuration (Chapter 22) rather than requiring a redesign.



25.1 Password Storage and Token Security

Passwords are hashed with **bcrypt** via `passlib`'s `CryptContext` (never stored or logged in plain text), and sessions are authenticated with **HS256-signed JWTs** carrying a 24-hour expiry. Notably, `config.py` contains *no* insecure fallback for `SECRET_KEY` — the application raises a startup `ValueError` and refuses to run if the environment variable is unset, which is good practice: it makes an accidentally-default signing key structurally impossible rather than merely discouraged.

25.2 Authorization

Every session-, message-, and ticket-scoped endpoint checks resource ownership against the authenticated user's ID before returning data, and — as noted in Chapter 17.6 — an unauthorized access attempt returns **404** rather than **403**, avoiding confirmation that a given resource ID exists at all to a caller who doesn't own it.

25.3 Input Validation and Injection Safety

All request bodies are validated against Pydantic v2 schemas before a route handler executes (Chapter 17.6) — including an explicit 1–2000 character bound on chat message length — and all database access goes through SQLAlchemy's ORM query interface, which parameterizes values and avoids raw string-interpolated SQL.

25.4 Transport and Secrets Management

CORS is restricted to an explicit origin allow-list rather than a wildcard (Chapter 13.1), and every credential-bearing environment variable in `render.yaml` — database URL, JWT secret, Groq/SMTP/Twilio credentials — is marked `sync: false`, meaning it must be supplied through Render's dashboard rather than committed to the repository. HTTPS termination itself is handled by the hosting platforms (Vercel and Render) rather than in application code, which is standard practice for this class of deployment.

25.5 Abuse Mitigation

The 20-message-per-minute rate limiter (Chapter 13.4) provides basic protection against a single account flooding the LLM/notification pipeline, with the multi-instance caveat already noted in Chapter 21.1.

25.6 Items to Address Before a Production Launch

Item	Current state	Recommendation
Default admin account	Seeded by <code>setup.py</code> and shown on the login page as a demo hint (Chapter 16.8)	Remove or rotate before any non-demo deployment
JWT storage location	<code>localStorage</code> on the client	Acceptable for this project's scope; an <code>httpOnly</code> cookie would reduce XSS-based token theft exposure in a hardened deployment
WhatsApp sender	Twilio sandbox number	Move to an approved production sender (Chapter 18.2)

⚠ SCOPE OF THIS CHAPTER

This is a description of implemented mechanisms and honest gaps, not a penetration-test report or a compliance audit. No security scanning tooling was run as part of producing this report.



26.1 Latency Budget

Retrieval itself is not the bottleneck: an exact search over 217 384-dimension vectors (Chapter 9.2) completes in low single-digit milliseconds. The dominant cost in the measured ~1.2-second average (Chapter 19.1) is LLM inference — even with Groq's LPU hardware chosen specifically for low-latency serving (Chapter 5), generating up to several hundred tokens through an 8-billion-parameter model remains the largest single component of end-to-end response time.

26.2 The Production Trade-off

The three production overrides documented in Chapter 18.4 — `TOP_K_RESULTS=1`, `MAX_TOKENS=150`, `TEMPERATURE=0.4` — are a direct, deliberate latency-versus-richness trade-off: a single retrieved chunk instead of four shortens the prompt, a 150-token cap shortens generation, and lower temperature reduces variance in generation length. This is the correct call for a free-tier deployment where perceived responsiveness matters more than maximal answer detail, and is disclosed here rather than left implicit.

26.3 Two Latency Regimes

There are effectively two different response-time experiences depending on backend process state: a **warm** request, matching the ~0.8–1.8 second range recorded in the Chapter 19 benchmark, and a **cold-start** request following a period of inactivity on Render's free tier, which can take on the order of a minute purely for the process to resume before any request-handling latency is even incurred (Chapter 18.5).

26.4 Scalability Ceiling of the Current Design

At present scale (217 chunks, a single backend process, SQLite by default), the system comfortably serves low-to-moderate concurrent traffic. The specific components that would need to change before materially higher scale — the rate limiter, the default database, and the flat vector index — are the same three already identified honestly in Chapter 21.1, each with a concrete, non-speculative upgrade path described in Chapter 22.

TAKEAWAY

Performance in this system is overwhelmingly an LLM-inference-latency problem, not a retrieval or database problem — which is exactly why the production tuning pass (Chapter 18.4) targets prompt size and generation length rather than the retrieval layer.

27



Conclusion

What this project demonstrates, stated plainly

This report has documented, chapter by chapter and against the actual delivered source code, a working Retrieval-Augmented Generation system that routes customer messages across five specialized agents, grounds every generated answer in TechMart Electronics' own policy and product documents, reacts automatically to detected frustration, and exposes all of that behavior through a fully-featured, deployed web application. Nothing in Chapters 6 through 19 — the technical core of this document — describes a planned or aspirational capability; every pipeline, every endpoint, and every measured figure traces to a specific file, function, or recorded evaluation artifact in the repository.

Measured against the objectives set out in Chapter 3, the implementation succeeds on its central engineering claim: retrieval-grounded, multi-agent responses measurably outperform what a single ungrounded prompt could offer, at a 90% classification accuracy and roughly 1.2-second response time that were *verified*, not asserted (Chapter 19). Equally, this report has not smoothed over what the system does not yet do — Chapter 21 catalogs real, specific gaps, from a documented-but-unfixed prompt-construction bug to the absence of an automated test suite — because an accurate account of a system's limitations is as much a demonstration of engineering maturity as its features are.

As a piece of applied engineering, the project's core contribution is showing that a small, focused team — in this case, a single developer during an internship — can take a RAG-based multi-agent architecture from source documents to a deployed, evaluated, publicly reachable application using entirely open or low-cost components: an open sentence-embedding model, an open-source vector index, a fast open-weight LLM, and free-tier hosting. That combination is what makes the architecture documented here directly transferable to the other industrial contexts outlined in Chapter 23, and it is offered in this report as a complete, honest technical reference for exactly that purpose.



Listed in order of first substantive use in this report. All entries correspond to frameworks, models, platforms, or datasets directly present in the delivered codebase.

- [1] FastAPI, "FastAPI framework, high performance web framework for building APIs with Python," *FastAPI Documentation*. [Online]. Available: <https://fastapi.tiangolo.com>
- [2] Meta AI, "Introducing Llama 3.1: Our most capable models to date," *Meta AI Blog*, 2024.
- [3] Groq, Inc., "GroqCloud API Documentation," *Groq Documentation*. [Online]. Available: <https://console.groq.com/docs>
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [5] Hugging Face, "sentence-transformers/all-MiniLM-L6-v2," *Hugging Face Model Hub*. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [6] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, 2019 — describing the FAISS library used for vector indexing and retrieval.
- [7] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [8] Vercel Inc., "Next.js Documentation," *Next.js*. [Online]. Available: <https://nextjs.org/docs>
- [9] Meta Platforms, Inc., "React Documentation," *React*. [Online]. Available: <https://react.dev>
- [10] Tailwind Labs, "Tailwind CSS Documentation," *Tailwind CSS*. [Online]. Available: <https://tailwindcss.com/docs>
- [11] SQLAlchemy authors, "SQLAlchemy 2.0 Documentation," *SQLAlchemy*. [Online]. Available: <https://docs.sqlalchemy.org>
- [12] Pydantic Services Inc., "Pydantic v2 Documentation," *Pydantic*. [Online]. Available: <https://docs.pydantic.dev>
- [13] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, 2015 — the token format implemented via the `python-jose` library.
- [14] Twilio Inc., "WhatsApp API for Business — Sandbox Documentation," *Twilio Docs*. [Online]. Available: <https://www.twilio.com/docs/whatsapp>
- [15] Twilio SendGrid, "SendGrid Web API v3 Documentation," *SendGrid Docs*. [Online]. Available: <https://docs.sendgrid.com>
- [16] I. Casanueva, T. Temčinas, D. Gerz, M. Henderson, and I. Vulić, "Efficient Intent Detection with Dual Sentence Encoders," in *Proc. 2nd Workshop on NLP for ConvAI, ACL, 2020* — source of the Banking77 corpus bundled in `datasets/`.
- [17] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation." [Online]. Available: <https://www.postgresql.org/docs>
- [18] Vercel Inc., "Vercel Platform Documentation." [Online]. Available: <https://vercel.com/docs>
- [19] Render Services, Inc., "Render Documentation — Web Services & Blueprints." [Online]. Available: <https://render.com/docs>

[20] pyca/passlib maintainers, "Passlib Documentation — CryptContext and bcrypt," *Passlib*. [Online]. Available: <https://passlib.readthedocs.io>